

Langage Go

Laurent Bellon

2025-2026

1-1-1-1

Google, Rob Pike, Ken Thompson – pourquoi Go en 2009

La Naissance d'un Langage Stratégique

- **Qui ?** Robert Griesemer, Rob Pike et Ken Thompson, ingénieurs chez Google.
- **Quand ?** Amorcé en 2007, Go est apparu en 2009.
- **Pourquoi ?** Répondre à un besoin précis de Google : améliorer la productivité des développeurs face à la complexité croissante des systèmes modernes.
- **L'objectif :** Créer un langage simple, efficace et performant pour le développement à grande échelle.

1-1-1-3

Les Solutions Apportées par Go

Les Objectifs Clés de Go

Go a été conçu pour surmonter ces obstacles avec des caractéristiques fondamentales :

- **Compilé et rapide** : Compilation rapide, même pour des projets massifs.
- **Syntaxe simple et lisible** : Inspirée de C, mais épurée de ses complexités inutiles.
- **Gestion native de la concurrence** : Intégrée dès la conception, basée sur le modèle CSP.
- **Garbage collection** : Simplifie la gestion mémoire tout en maintenant des performances élevées.
- **Un écosystème intégré** : Un ensemble d'outils standards (compilateur, formateur, gestionnaire de paquets) pour une expérience de développement cohérente.

1-1-1-5

Le Modèle de Concurrency de Go : CSP

Communiquer Plutôt que Partager

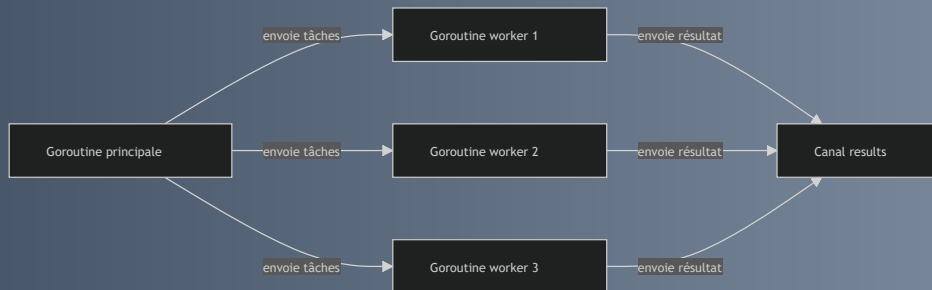
Go s'inspire du modèle **Communicating Sequential Processes (CSP)**, établi en 1978 :

- **Principe** : Les programmes sont composés de processus simples.
- **Communication** : Ces processus communiquent via des **canaux**.
- **Avantage** : Permet d'écrire des programmes concurrents sans les complexités des verrous ou des sections critiques.
- **Philosophie** : "Ne pas communiquer en partageant la mémoire ; partager la mémoire en communiquant."

1-1-1-7

Visualisation du Modèle Concurrent de Go

Flux de Travail avec Goroutines et Canaux



- La **Goroutine principale** distribue les tâches.
- Les **Goroutines worker** traitent les tâches en parallèle.
- Le **Canal results** collecte les résultats de manière sécurisée.

1-1-1-8

Conclusion : L'Impact Durable de Go

- **Réponse à un besoin** : Go a été créé pour fournir à Google un langage moderne, performant, simple et adapté à la construction de systèmes à grande échelle et concurrentiels.
- **Synthèse d'idées** : Il combine la simplicité de C, l'efficacité du modèle CSP et une forte orientation vers la productivité des développeurs.
- **Positionnement actuel** : Go est devenu un langage incontournable pour :
 - Les systèmes distribués
 - Les infrastructures cloud
 - Les microservices

1-2-1-2

Performance : Le Classement des Langages

- **C** : Compilé bas niveau, contrôle fin du matériel, performances maximales. Gestion manuelle de la mémoire, source d'erreurs.
- **Rust** : Sécurité mémoire via système de propriété (sans GC), performances très proches de C.
- **Go** : Compilé en code machine avec un ramasse-miettes (GC). Facilite la gestion mémoire, mais légère perte de performances par rapport à C/Rust.
- **Java** : Compile en bytecode pour JVM, GC performant mais coûteux en latence.
- **Python** : Interprété, peu performant en vitesse d'exécution, mais apprécié pour sa simplicité.

Résumé performance (de la plus rapide à la plus lente) : C ≈ Rust > Go > Java > Python

1-2-1-3

Simplicité et Productivité : Python et Go

- **Python** : Réputé pour sa syntaxe simple et sa rapidité de prototypage (scripts, data science).
- **Go** : Partage cette orientation vers la simplicité :
 - Syntaxe claire et restreinte (moins de concepts que Java ou C++).
 - Pas d'héritage multiple ni de génériques complexes.
 - Gestion de la concurrence explicite et simple (goroutines et canaux).
 - Outils intégrés pour formatage, documentation, gestion des paquets (modules).
- **Java** et **Rust** sont plus verbeux et complexes, avec des courbes d'apprentissage plus longues.

1-2-1-4

Concurrence et Parallélisme

Go propose un modèle de **concurrency légère** via :

- **Goroutines** : Fonctions légères s'exécutant en parallèle, multiplexées sur peu de threads système.
- **Canaux (channels)** : Mécanismes de communication sûrs entre goroutines, basés sur le modèle CSP.

Ce modèle facilite l'écriture d'applications concurrentes par rapport aux threads classiques de Java ou aux paradigmes bas niveau de C.

- **Rust** : Concurrency sûre grâce à son système de possession et de références, mais avec plus de complexité.
- **Python** : La Global Interpreter Lock (GIL) limite la concurrence réelle dans les threads.

1-2-1-5

Gestion Mémoire Comparée

Langage	Gestion mémoire	Avantages	Limites
C	Manuelle	Contrôle maximum	Erreurs fréquentes (fuites, accès invalides)
Rust	Automatique via ownership	Sécurité et performance	Courbe d'apprentissage élevée
Go	Garbage collector	Facilité d'utilisation	Pause GC pouvant impacter les latences
Java	Garbage collector	Maturité et optimisation	Gestion mémoire lourde pour certains cas
Python	Relevé de compte + GC	Simple d'utilisation	Lenteur, GIL limite la vraie concurrence

1-2-1-6

Exemple de Code : Simplicité et efficacité

Fonction calculant la somme des entiers de 1 à n

Go

```
func sum(n int) int {  
    total := 0  
    for i := 1; i ≤ n; i++ {  
        total += i  
    }  
    return total  
}
```

C

```
int sum(int n) {  
    int total = 0;  
    for (int i = 1; i ≤ n; i++) {  
        total += i;  
    }  
    return total;  
}
```

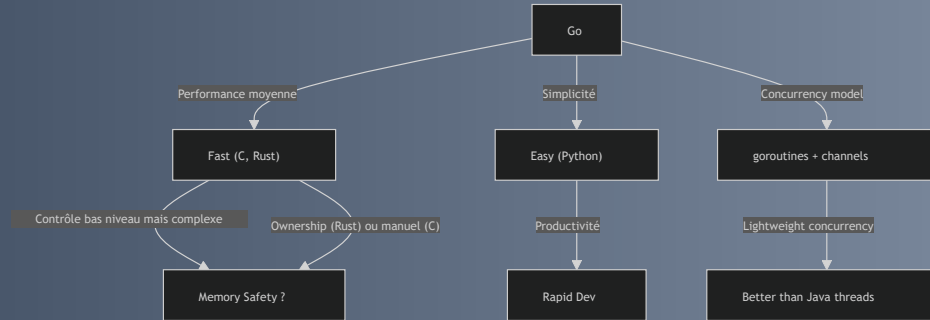
Python

```
def sum(n):  
    total = 0  
    for i in range(1, n+1):  
        total += i  
    return total
```

Cet exemple illustre la proximité de Go avec la simplicité d'écriture de Python, tout en restant un langage compilé et performant.

1-2-1-7

Synthèse Visuelle des Langages



1-2-1-8

Conclusion et Cas d'Usage

Go offre un équilibre unique :

- Meilleure performance que Python et Java.
- Syntaxe plus simple que Rust et C++.
- Gestion concurrente intégrée et outils standardisés facilitent le développement moderne (systèmes distribués, services réseau).

Quand choisir Go ? Quand la productivité, la simplicité et une bonne performance globale sont recherchées.

Quand choisir C ou Rust ? Pour des applications nécessitant une performance au plus haut niveau et un contrôle mémoire strict.

1-3-1-1

Téléchargement, Installation et Configuration Go

GOROOT & GOPATH

Introduction

Pour développer en Go, une installation correcte et une compréhension des variables clés **GOPATH** et **GOROOT** sont essentielles.

Cette présentation vous guide à travers les étapes d'installation, de configuration et de création de votre premier programme Go.

Télécharger et Installer Go

Étapes d'installation

1. Téléchargement officiel

1. Visitez <https://golang.org/dl/>
2. Choisissez la version stable adaptée à votre OS (ex: `go1.20.5.linux-amd64.tar.gz`).

2. Installation sous Linux / macOS

```
sudo tar -C /usr/local -xzf go1.20.5.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin
source ~/.bashrc # ou ~/.zshrc
```

3. Installation sous Windows

1. Téléchargez l'installeur `.msi` et suivez les instructions. Le PATH est automatiquement configuré.

1-3-1-3

GOROOT : L'Emplacement de Go

Définition

- **Définit l'emplacement de l'installation du langage Go.**
- Contient le compilateur, la bibliothèque standard, etc.
- **Emplacement typique :**
 - `/usr/local/go` (Linux/macOS)
 - `C:\Go` (Windows)
- **Ne doit pas être modifié** sauf cas avancés.

Vérification

```
go env GOROOT
```

GOPATH : Votre Espace de Travail Go

Définition

- **Définit le répertoire de travail pour vos projets Go, packages, binaires.**
- **Par défaut :**
 - `$HOME/go` (Linux/macOS)
 - `%USERPROFILE%\go` (Windows)

Structure typique

```
GOPATH/  
├─ bin/      # exécutables compilés  
├─ pkg/      # packages compilés  
└─ src/      # source des projets
```

Configuration (ex: `.bashrc`)

```
export GOPATH=$HOME/go  
export PATH=$PATH:$GOPATH/bin
```

Vérification

```
go env GOPATH
```

1-3-1-5

Premier Programme Go : "Bonjour, Go !"

Création de `hello.go`

```
package main

import "fmt"

func main() {
    fmt.Println("Bonjour, Go !")
}
```

Placez ce fichier sous `$GOPATH/src/helloworld/`.

Compilation et exécution

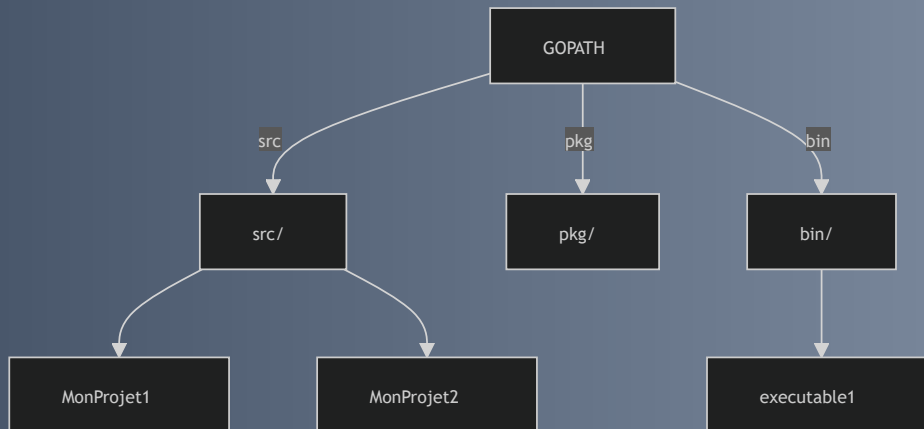
```
cd $GOPATH/src/helloworld  
go mod init hello  
go build  
./helloworld
```

Affiche : Bonjour, Go !

Exécution directe sans build

```
go run hello.go  
go run .
```

Organisation avec GOPATH (Avant les Modules)



- **src** : Contient le code source de vos projets.
- **pkg** : Stocke les packages compilés.
- **bin** : Contient les exécutables générés.

Note : Depuis Go 1.11, les modules (`go mod`) ont réduit la dépendance à GOPATH pour la gestion des dépendances.

1-3-1-7

Vérification de l'Installation

Confirmer la version de Go

```
go version
```

Afficher toutes les variables d'environnement Go

```
go env
```

Ces commandes vous permettent de valider que Go est correctement installé et configuré sur votre système.

1-3-1-8

Conclusion & Ressources

En résumé

- L'installation de Go est simple et rapide.
- **GOROOT** est l'emplacement de l'installation de Go (ne pas modifier).
- **GOPATH** définit votre espace de travail pour les projets (moins critique avec les modules Go 1.11+).
- Un premier programme permet de valider votre configuration.

Sources

- [Go.dev - Installation](#)
- [Go.dev - Environment Variables](#)
- [GOPATH vs GOROOT - Stack Overflow](#)
- [Go Wiki - Setting GOPATH](#)

1-3-2-1

Premier programme 'Hello World' en Go

Le point de départ en programmation

- Le programme "Hello World" est la première étape pour découvrir un nouveau langage.
- En Go, il est simple, lisible et illustre des concepts fondamentaux.

Ce que nous allons explorer :

- La structure d'un package Go.
- L'importation de bibliothèques.
- La fonction d'entrée `main`.

1-3-2-2

Anatomie d'un Programme Go

1. Déclaration du package Chaque programme Go démarre par une déclaration de package. Pour les exécutables, c'est `package main`.

```
package main
```

2. Importation de bibliothèques Les packages standard ou tiers sont importés. `fmt` est utilisé pour l'affichage.

```
import "fmt"
```

3. La fonction principale `main` C'est le point d'entrée obligatoire de votre programme.

```
func main() {  
    // Votre code ici  
}
```

1-3-2-3

Votre Premier Code "Hello World"

Le fichier `hello.go` complet :

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

Décryptage ligne par ligne :

- `package main` : Indique que c'est une application exécutable.
- `import "fmt"` : Permet d'utiliser la bibliothèque standard `fmt` (formatage).
- `func main()` : La fonction obligatoire d'exécution du programme.
- `fmt.Println` : Affiche le texte "Hello, World!" suivi d'un retour à la ligne.

1-3-2-4

Compiler votre Programme Go

Étape 1 : Placez-vous dans le dossier Ouvrez votre terminal et naviguez jusqu'au répertoire contenant `hello.go`.

Étape 2 : Initialisez le code Utilisez la commande `go mod init hello` pour générer un exécutable.

```
go mod init hello
```

Résultat : Un fichier `go.mod` est créé.

Étape 2 : Compilez le code Utilisez la commande `go build` pour générer un exécutable.

```
go build hello.go
```

Résultat : Un fichier `hello` (ou `hello.exe` sur Windows) est créé.

Étape 3 : Exécutez le programme Lancez l'exécutable généré.

```
./hello
```

Sortie attendue :

```
Hello, World!
```

1-3-2-5

Exécution Directe Simplifiée

Gagnez du temps avec `go run`

- Go offre une manière plus rapide d'exécuter vos programmes sans compilation explicite préalable.
- La commande `go run` compile et lance le programme en une seule étape.

Comment faire ?

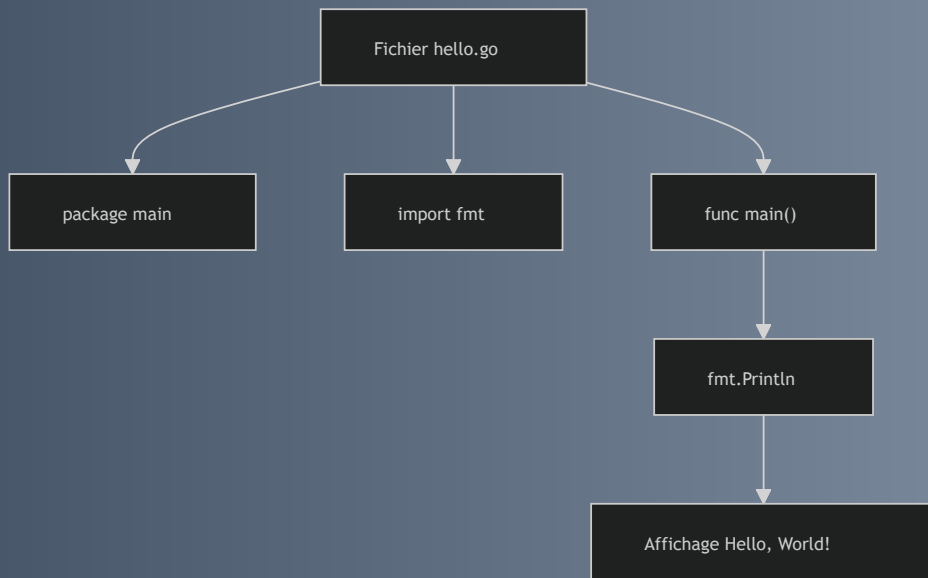
```
go run hello.go
```

Sortie attendue :

```
Hello, World!
```

Cette méthode est idéale pour tester rapidement de petits programmes.

Comprendre le Flux d'Exécution



- Le compilateur Go identifie `func main()` comme le point d'entrée.
- Le package `fmt` fournit la fonction `Println` pour l'affichage.
- L'exécutable est généré et lié grâce à la commande `go build`.

1-3-2-7

Remarques Essentielles sur Go

Points clés à retenir :

- **Package** `main` : Tout programme Go produisant un exécutable doit définir un package `main`.
- **Fonction** `main` : Elle ne prend ni arguments, ni ne renvoie de valeur.
- `fmt.Println` : Ajoute automatiquement un saut de ligne après le texte affiché.
- **Système de paquets** : Go encourage une structuration propre des projets, même les plus complexes.

1-3-2-8

Étendre Votre Programme & Prochaines Étapes

Exemple d'extension simple : Pour afficher plusieurs messages, il suffit d'ajouter des appels à `fmt.Println`.

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
    fmt.Println("Bienvenue dans le langage Go.")
}
```

Exécution :

```
Hello, World!
Bienvenue dans le langage Go.
```


Conclusion : Ce premier programme vous a permis d'appréhender les bases syntaxiques et l'environnement de développement de Go. Il pose les fondations pour écrire rapidement des applications plus complexes.

Sources :

- [Documentation officielle Go - Hello World](#)
- [Tour of Go - Basics](#)
- [Go by Example - Hello World](#)
- [Go Command Documentation](#)

1-4-1-1

Go Modules : La Révolution de la Gestion de Projet

- **Contexte** : Depuis Go 1.11, un nouveau système de gestion de dépendances basé sur les **modules** a remplacé l'ancien GOPATH.
- **Objectif** : Simplifier la gestion des dépendances, le versionnage et la reproductibilité des builds.
- **Cette présentation explore** :
 - Les **packages** (unités d'organisation).
 - Les **modules** (gestion des dépendances via `go.mod`).
 - Les **workspaces** (`go.work`).
 - Les commandes clés pour un projet Go moderne.

Packages : L'Unité d'Organisation du Code

- Un **package** est un ensemble de fichiers `.go` regroupés sous un même nom.
- Il sert à organiser le code source en unités fonctionnelles réutilisables.
- Chaque fichier `.go` commence par :

```
package nomDuPackage
```

- Le package `main` est spécial : il produit un exécutable avec une fonction `main()` d'entrée.

Exemple (package `utils`) :

```
package utils

func Add(a, b int) int {
    return a + b
}
```

Importation :

```
import "monprojet/utils"
```

1-4-1-3

Modules : Le Cœur de la Gestion des Dépendances

- Un **module** est un ensemble de packages versionné, défini par un répertoire racine contenant un fichier `go.mod`.
- Le fichier `go.mod` décrit :
 - Le nom (path) du module.
 - Les dépendances externes (autres modules) avec leurs versions.
 - Les contraintes de versionnage.
- Il permet à Go de résoudre les dépendances et versions à utiliser.

Initialisation d'un module : Dans le répertoire racine du projet :

```
go mod init monmodule/nomprojet
```

Ceci crée un fichier `go.mod`

1-4-1-4

go.mod : Le Manifeste des Dépendances

Le fichier `go.mod` liste les modules requis et leurs versions exactes.

Exemple détaillé :

```
module github.com/user/monprojet

go 1.20

require (
    github.com/some/dependency v1.2.3
    golang.org/x/text v0.4.0
)
```

- `module` : Nom unique du module.
- `go` : Version minimale de Go requise.
- `require` : Liste des dépendances externes avec leurs versions.
- Ce fichier est auto-généré et mis à jour par les commandes Go.

1-4-1-5

Workspaces (`go.work`) : Collaboration Multi-Modules

Introduit pour gérer plusieurs modules localement dans un même workspace. Utile pour le développement simultané de plusieurs projets liés.

Création d'un workspace :

```
go work init ./moduleA ./moduleB
```

Génère un fichier `go.work` :

```
go 1.20

use (
  ./moduleA
  ./moduleB
)
```

- `go` : Version minimale de Go pour le workspace.
- `use` : Liste des chemins vers les modules inclus dans le workspace.

1-4-1-6

Commandes Clés pour Gérer Modules et Packages

Commande	Description
<code>go mod init</code>	Initialise un nouveau module Go
<code>go mod tidy</code>	Nettoie les dépendances inutilisées ou manquantes.
<code>go mod download</code>	Télécharge les dépendances décrites dans go.mod
<code>go get [paquet@version]</code>	Ajoute ou met à jour une dépendance
<code>go mod verify</code>	Vérifie l'intégrité des modules téléchargés
<code>go work init [dirs]</code>	Crée un workspace liant plusieurs modules

1-4-1-7

Exemple Concret : Structure et Code

Organisation d'un projet simple :

```
monprojet/  
├── go.mod  
├── main.go  
├── utils/  
│   └── utils.go  
└── go.work (optionnel, pour plusieurs modules)
```


main.go :

```
package main

import (
    "fmt"
    "monprojet/utils" // Importe le package local "utils"
)

func main() {
    result := utils.Add(2, 3)
    fmt.Println("Resultat:", result)
}
```

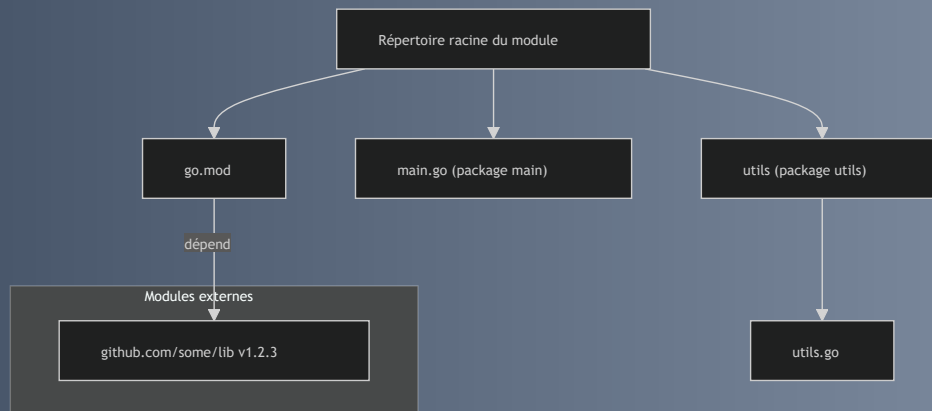
utils/utils.go :

```
package utils

func Add(a, b int) int {
    return a + b
}
```

1-4-1-8

Organisation Visuelle et Conclusion



En résumé : Le système de modules Go avec `go.mod` a simplifié la gestion des dépendances et amélioré la modularité. Comprendre les packages, les modules et le workspace permet d'organiser efficacement son code, gérer les versions des dépendances et garantir portabilité et reproductibilité des builds.

Types de Base et Inférence en Go

- Les Types Entiers en Go

Type	Taille	Description
<code>int</code>	32 ou 64 bits*	Taille dépend de l'architecture
<code>int8</code>	8 bits	Entier signé
<code>int16</code>	16 bits	Entier signé
<code>int32</code>	32 bits	Entier signé
<code>int64</code>	64 bits	Entier signé
<code>uint</code>	32 ou 64 bits*	Entier non signé
<code>uint8</code>	8 bits	Alias de byte
<code>uint16</code>	16 bits	Entier non signé
<code>uint32</code>	32 bits	Entier non signé
<code>uint64</code>	64 bits	Entier non signé

2-1-1-3

Types Flottants, Booléens et Chaînes

Types flottants (floats) : Pour les nombres à virgule flottante :

- `float32` : précision simple (32 bits)
- `float64` : précision double (64 bits), recommandé pour la plupart des usages.

Exemple :

```
var f1 float32 = 3.14  
var f2 float64 = 2.718281828
```

Type booléen : Prend uniquement deux valeurs : `true` ou `false` .

```
var flag bool = true
```

Une chaîne de caractères UTF-8 immuable.

Exemple :

```
var s string = "Bonjour, Go!"
```

2-1-1-4

Inférence Automatique de Type

Go propose une syntaxe de déclaration simplifiée via l'opérateur `:=` qui infère automatiquement le type depuis la valeur à droite.

Exemple :

```
a := 42           // int
b := 3.14         // float64
flag := true      // bool
s := "Hello"      // string
```

Note : Cette déclaration ne peut être utilisée qu'à l'intérieur d'une fonction.

2-1-1-5

Les Constantes

Les constantes sont déclarées avec le mot-clé `const`.

Caractéristiques :

- Elles doivent être évaluées à la compilation.
- Elles ne peuvent pas être modifiées après leur déclaration.

Exemples :

```
const Pi = 3.14159  
const Greeting = "Bonjour"  
const IsActive = true
```

2-1-1-6

Exemple Complet

```
package main

import "fmt"

func main() {
    var i int = 10
    j := 20.5          // inférence float64
    var isReady bool = true
    msg := "Bonjour Go"

    fmt.Println("i =", i)
    fmt.Println("j =", j)
    fmt.Println("isReady =", isReady)
    fmt.Println("msg =", msg)
}
```


Catégorisation des Types de Base Go

2-1-1-8

Conclusion et Références

Cette présentation vous a donné une vision des types primitifs classiques en Go, ainsi que des mécanismes d'inférence typée et des constantes. Ces bases sont indispensables pour écrire un code sûr, clair et performant.

Références :

- [Go by Example - Variables](#)
- [Go Language Spec - Basic Types](#)
- [Effective Go - Variables](#)
- [Tour of Go - Values](#)
- [Go Wiki - FAQ Type System](#)

2-2-1-1

Structures de contrôle en Go : `if`, `for`, `switch`, `defer`

Introduction

Les structures de contrôle sont les mécanismes fondamentaux qui dirigent le flux d'exécution d'un programme. Go offre un ensemble simple et puissant :

- `if` : Pour la conditionnelle.
- `for` : Pour les boucles (la seule structure de boucle).
- `switch` : Pour les sélections multiples.
- `defer` : Pour la gestion différée d'instructions, très utile pour la gestion des ressources.

if : La conditionnelle en Go

Syntaxe et usages

- Syntaxe classique avec ou sans accolades.
- La condition doit être une expression booléenne.
- Go autorise une instruction d'init avant la condition.

Exemple sans init :

```
if x > 10 {  
    fmt.Println("x est plus grand que 10")  
}
```

Exemple avec init (gestion d'erreur courante) :

```
if err := doSomething(); err != nil {  
    fmt.Println("Erreur détectée :", err)  
}
```

`for` : La boucle unique et flexible

Une seule structure pour toutes les boucles

`for` est la seule structure de boucle en Go, capable d'exprimer :

- **Boucle classique** (initialisation, condition, post-incrément)
- **Boucle style** `while` (condition seule)
- **Boucle infinie** (sans condition)

Les variables d'itération sont locales à la boucle.

Boucle classique :

```
for i := 0; i < 5; i++ {  
    fmt.Println(i)  
}
```

Boucle style while :

```
i := 0
for i < 5 {
    fmt.Println(i)
    i++
}
```

Boucle infinie :

```
for {
    // Exécution répétée
}
```

2-2-1-4

`switch` : Sélection multiple simplifiée

Tests multiples sans `break`

- Permet des tests multiples sur une valeur.
- **Pas besoin de** `break` **explicite**, Go sort automatiquement du switch après un case.
- Supporte les expressions dans les cases.
- Peut s'écrire sans expression ; dans ce cas, les cases sont évaluées comme des conditions booléennes.

Switch sur valeur :

```
switch day {  
case "Monday":  
    fmt.Println("Début de semaine")  
case "Friday":  
    fmt.Println("Fin de semaine")  
default:  
    fmt.Println("Autre jour")  
}
```

Switch sans expression (conditions booléennes) :

```
switch {  
case x < 0:  
    fmt.Println("Négatif")  
case x == 0:  
    fmt.Println("Zéro")  
default:  
    fmt.Println("Positif")  
}
```


`defer` : Exécution différée et gestion des ressources

Assurer la propreté du code

- Permet de différer l'exécution d'une fonction jusqu'à la fin de la fonction englobante.
- Très utile pour fermer des fichiers, libérer des ressources, afficher des logs de fin.
- Plusieurs `defer` sont empilés et exécutés dans l'ordre inverse de leur déclaration (LIFO : Last In, First Out).

Exemple simple :

```
func readFile() {  
    file, err := os.Open("test.txt")  
    if err != nil {  
        fmt.Println("Erreur ouverture fichier:", err)  
        return  
    }  
    defer file.Close() // Sera exécuté à la fin de readFile()  
    // traitement du fichier ici  
    fmt.Println("Lecture terminée")  
}
```

Structures combinées

```
package main

import (
    "fmt"
    "os"
)

func main() {
    // Démonstration de defer (LIFO)
    for i := 0; i < 3; i++ {
        defer fmt.Println("Deferred:", i) // S'affiche dans l'ordre 2, 1, 0
    }

    // if avec instruction d'init
    if err := doSomething(); err != nil {
        fmt.Println("Erreur :", err)
    }

    // switch sur valeur
    switch x := 2; x {
    case 1:
        fmt.Println("Un")
    case 2:
        fmt.Println("Deux")
    default:
```

```
func doSomething() error {  
    file, err := os.Open("file.txt") // Supposons que file.txt n'existe pas  
    if err != nil {  
        return err // Retourne l'erreur  
    }  
    defer file.Close() // Ne sera pas exécuté si une erreur survient avant  
    // Traitement fichier...  
    return nil  
}
```

L'exécution de `defer` affiche les sorties dans l'ordre inverse (2, 1, 0).

Simplicité et puissance du contrôle de flux

En bref & Ressources

Les structures de contrôle `if`, `for`, `switch` fournissent une base solide et expressive pour piloter l'exécution d'un programme Go, tout en restant simples et lisibles. Le mot-clé `defer` ajoute une grande flexibilité pour la gestion sûre des ressources.

Sources :

- [Go by Example - If/Else](#)
- [Go by Example - For](#)
- [Go by Example - Switch](#)
- [Go by Example - Defer](#)
- [Golang official documentation - Flow control](#)
- [Effective Go - Defer](#)

2-3-1-1

Fonctions avec valeurs multiples, variadiques, fonctions comme variables en Go

Introduction aux Fonctions Go

Les fonctions en Go offrent des fonctionnalités avancées tout en restant simples à utiliser.

Parmi leurs caractéristiques puissantes :

- La capacité à retourner plusieurs valeurs.
- L'acceptation d'un nombre variable d'arguments (variadique).
- La manipulation des fonctions comme des valeurs (variables, arguments, retours).

Cet article aborde ces concepts clés avec des exemples concrets.

2-3-1-2

1. Fonctions à valeurs multiples

Go permet à une fonction de retourner plusieurs résultats.

Utilité :

- Retourner une valeur et une erreur.
- Retourner plusieurs résultats complémentaires.

Cette capacité évite souvent le recours à des structures ou à l'usage d'exceptions.

Exemple : fonction division retournant quotient et reste

```
func divMod(a, b int) (int, int) {  
    return a / b, a % b  
}
```

Utilisation :

```
q, r := divMod(10, 3)  
fmt.Println("Quotient:", q, "Reste:", r) // Affiche: Quotient: 3 Reste: 1
```

2-3-1-3

2. Fonctions variadiques

Une fonction variadique accepte un nombre variable d'arguments du même type.

Le dernier paramètre est précédé de `...`.

Exemple : somme de plusieurs entiers

```
func sum(nums ...int) int {  
    total := 0  
    for _, n := range nums {  
        total += n  
    }  
    return total  
}
```


Appel possible avec n arguments :

```
fmt.Println(sum(1, 2, 3))    // Affiche: 6  
fmt.Println(sum(10, 20))    // Affiche: 30
```

Passer explicitement une slice :

```
vals := []int{5,6,7}  
fmt.Println(sum(vals ...))   // Affiche: 18
```

3. Fonctions comme variables (fonctions de première classe)

En Go, les fonctions sont des valeurs. Elles peuvent être :

- Affectées à des variables.
- Passées en paramètre.
- Retournées depuis d'autres fonctions.

Exemple : Affectation à une variable

```
func add(a, b int) int {  
    return a + b  
}  
  
func main() {  
    var f func(int, int) int // Déclaration d'une variable de type fonction  
    f = add                  // Affectation de la fonction 'add' à 'f'  
    fmt.Println(f(3, 4))    // Affiche: 7  
}
```

Exemple : Passage de fonction en argument

```
func apply(op func(int,int)int, x, y int) int {  
    return op(x, y)  
}
```

```
result := apply(add, 5, 6)  
fmt.Println(result) // Affiche: 11
```

4. Exemple complet mêlant les notions

```
package main

import "fmt"

func divMod(a, b int) (int, int) {
    return a / b, a % b
}

func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}
```

```
func apply(op func(int, int) int, x, y int) int {
    return op(x, y)
}

func add(a, b int) int {
    return a + b
}

func main() {
    q, r := divMod(20, 7)
    fmt.Println("divMod:", q, r) // divMod: 2 6

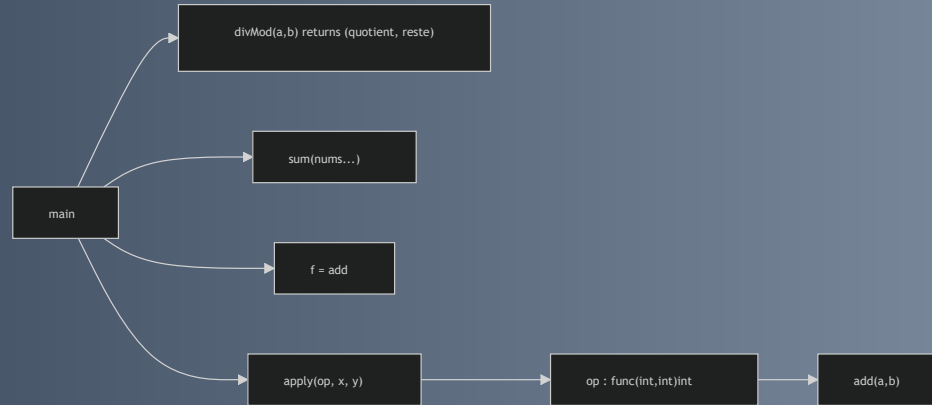
    fmt.Println("sum variadique:", sum(1, 2, 3, 4)) // sum variadique: 10

    f := add
    fmt.Println("fonction comme variable:", f(10, 15)) // fonction comme variable: 25

    result := apply(add, 5, 6)
    fmt.Println("fonction en argument:", result) // fonction en argument: 11
}
```

2-3-1-6

5. Diagramme relations fonctionnelles



2-3-1-7

6. Références

- [Go by Example - Multiple Return Values](#)
- [Go by Example - Variadic Functions](#)
- [Go by Example - Functions as Values](#)
- [Go Language Specification - Function types](#)
- [Effective Go - Functions](#)

2-4-1-1

Pointeurs en Go : Sécurité et Puissance

Introduction

- Le concept de pointeur est fondamental en langage système comme Go.
- Go adopte une approche simplifiée et sécurisée comparé à C.
- Cette présentation détaille l'usage des pointeurs en Go, le passage par référence et les différences clés avec le langage C.

Comprendre les Pointeurs en Go

Définition et usage

- Un pointeur est une variable qui contient l'adresse mémoire d'une autre variable.
- **Type** : `*T` (où `T` est un type).
- **Opérateur** `&` : Récupère l'adresse d'une variable.
- **Opérateur** `*` : Accède à la valeur pointée (déréférencement).

Exemple :

```
package main

import "fmt"

func main() {
    x := 42
    p := &x           // p est un pointeur vers x
    fmt.Println(*p)    // déréférencement : affiche 42

    *p = 21            // modifie la valeur pointée (x)
    fmt.Println(x)     // affiche 21
}
```

2-4-1-3

Modifier des variables via le Passage par Référence

Passage par référence avec pointeurs

- Par défaut, Go passe les arguments aux fonctions par copie (valeur).
- Pour modifier la variable d'origine, on utilise un pointeur.

Exemple :

```
func increment(n *int) {  
    *n = *n + 1  
}  
  
func main() {  
    x := 5  
    increment(&x)  
    fmt.Println(x)    // affiche 6  
}
```

2-4-1-4

Go vs C : Les Limites des Pointeurs en Go

Aspect	Go	C
Arithmétique de pointeur	Non supportée	Supportée
Pointeurs sur pointeurs	Supporté	Supporté
Pointeurs <code>void*</code>	Non existant (typage strict)	Existe (pointeur générique)
Manipulation mémoire brute	Restreinte	Libre (mais risque de corruption)
Garbage Collector	Oui (gestion automatique)	Non (gestion manuelle)

Quand et Comment Utiliser les Pointeurs

Conseils d'usage

- Utilisez les pointeurs pour éviter les copies coûteuses (ex : gros structs).
- Utilisez-les pour modifier les valeurs dans les fonctions.
- Évitez l'arithmétique de pointeurs en Go, privilégiez les structures et slices.
- La gestion mémoire est sécurisée, pas de `free` ou `malloc` explicite.

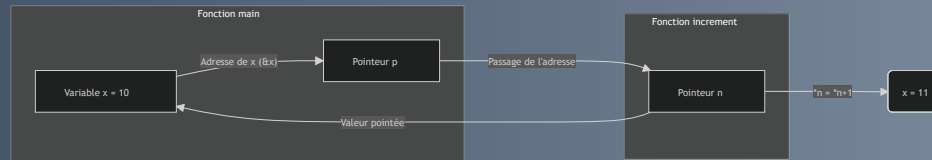
Exemple complet : échange de valeurs

```
func swap(a, b *int) {  
    temp := *a  
    *a = *b  
    *b = temp  
}  
  
func main() {  
    x, y := 3, 7  
    swap(&x, &y)  
    fmt.Println(x, y) // affiche 7 3  
}
```

2-4-1-6

Visualisation : Pointeur et Passage par Référence

Fonctionnement d'un pointeur et passage par référence



2-4-1-7

Go vs C : Récapitulatif des Différences Majeures

- **Pas d'arithmétique de pointeurs** : Go simplifie la sécurité mémoire.
- **Gestion automatique** via Garbage Collector : évite fuites liées aux pointeurs.
- **Typage plus strict** : pas de pointeurs génériques `void*`.
- **Syntaxe claire et sécurisée** utilisant `&` et `*` uniquement.

2-4-1-8

Conclusion & Pour Aller Plus Loin

Conclusion

- Go propose une approche moderne et simplifiée des pointeurs.
- Ils limitent les erreurs tout en offrant la puissance nécessaire pour la programmation système et la performance.
- Les pointeurs restent un outil indispensable quand on veut passer par référence ou éviter des copies inutiles.

Ressources

- [Go Blog - Pointers](#)
- [Go Language Spec - Pointers](#)
- [Go by Example - Pointers](#)
- [Effective Go - Pointers](#)
- [C vs Go Pointers Discussion](#)

2-5-1-1

Introduction aux collections en Go

Tableaux, slices et maps en Go - Manipulation, dimensionnement, allocation, pièges et optimisations

Go offre trois types fondamentaux pour gérer des collections de données :

- Les **tableaux (arrays)**
- Les **slices (tranches)**
- Les **maps**

Comprendre leurs différences, leur dimensionnement, ainsi que les pièges classiques et optimisations associées est indispensable pour écrire du code efficace et sûr.

2-5-1-2

1. Tableaux (Arrays) - La rigidité

Les tableaux sont le type de collection le plus basique en Go.

- **Taille fixe** déterminée à la compilation.
- **Syntaxe :** `[taille]T` où T est le type élémentaire.
- **Stockage contigu** en mémoire.
- **Peu utilisés directement** car rigides (taille fixe).
- **Copiés par valeur :** l'affectation ou passage en argument crée une copie entière du tableau.

Exemple :

```
var arr [3]int = [3]int{1, 2, 3}  
fmt.Println(arr) // Affiche: [1 2 3]
```

2. Slices (tranches) - La flexibilité (Partie 1)

Les slices offrent une vue dynamique et flexible sur un tableau sous-jacent.

- **Vue flexible** sur un tableau, dynamique en taille.
- Représentées par une structure interne avec :
 - Un **pointeur** sur le tableau sous-jacent
 - Une **longueur** (`len`)
 - Une **capacité** (`cap`)
- **Syntaxe** : `[]T` (sans taille).
- **Partagent la mémoire** sous-jacente avec d'autres slices.

Créer une slice :

```
s := []int{1, 2, 3} // Création à partir d'un littéral
```

Création via `make` **avec capacité :**

```
s := make([]int, 0, 5) // longueur 0, capacité 5
```

2. Slices (tranches) - `append` et Pièges (Partie 2)

La fonction `append` est clé pour manipuler les slices, mais attention aux effets de bord.

- `append` : Permet d'ajouter des éléments.
 - Si la capacité est dépassée, Go alloue un nouveau tableau plus grand (souvent double) et copie les données.

```
s = append(s, 4, 5)
```

- **Attention aux pièges :**
 - **Copier une slice** copie la structure (pointeur, len, cap) mais pas les données. Plusieurs slices peuvent partager la même mémoire.
 - **Modifier un élément** via une slice modifie le tableau sous-jacent, visible par d'autres slices pointant dessus.
 - **Redimensionner** (append au-delà de la capacité) crée une nouvelle allocation : la slice est alors dissociée de l'ancien tableau.

3. Maps - Clé-Valeur

Les maps sont des collections non ordonnées de paires clé-valeur.

- **Structure de données clé-valeur.**
- **Syntaxe :** `map[KeyType]ValueType` .
- **Allocation via `make` :**

```
m := make(map[string]int)
m["age"] = 30
```

- **Opérations courantes :** insert, accès, suppression (`delete`), test d'existence.

Piège : Accès à une map avec clé absente retourne la valeur zéro du type, ce qui peut induire en erreur si non contrôlé.

Test d'existence :

```
v, ok := m["key"]
if ok {
    // clé présente, v contient sa valeur
}
```

4. Dimensionnement et allocation

- **Tableaux :**

- Allocation statique, taille fixe.
- Usage adapté aux données de taille connue et constante.

- **Slices :**

- Allocation dynamique.
- Capacité et longueur contrôlent la croissance.
- **Optimisation :** Utiliser `make` avec une capacité estimée pour limiter les réallocations fréquentes.

```
s := make([]int, 0, 100) // capacité prévue à 100 éléments
```

- **Maps :**

- La capacité initiale ne peut être spécifiée directement, mais elle est ajustée dynamiquement.
- Pour de grandes maps, estimer la taille peut être utile pour limiter les réallocations internes.

2-5-1-7

5. Optimisations

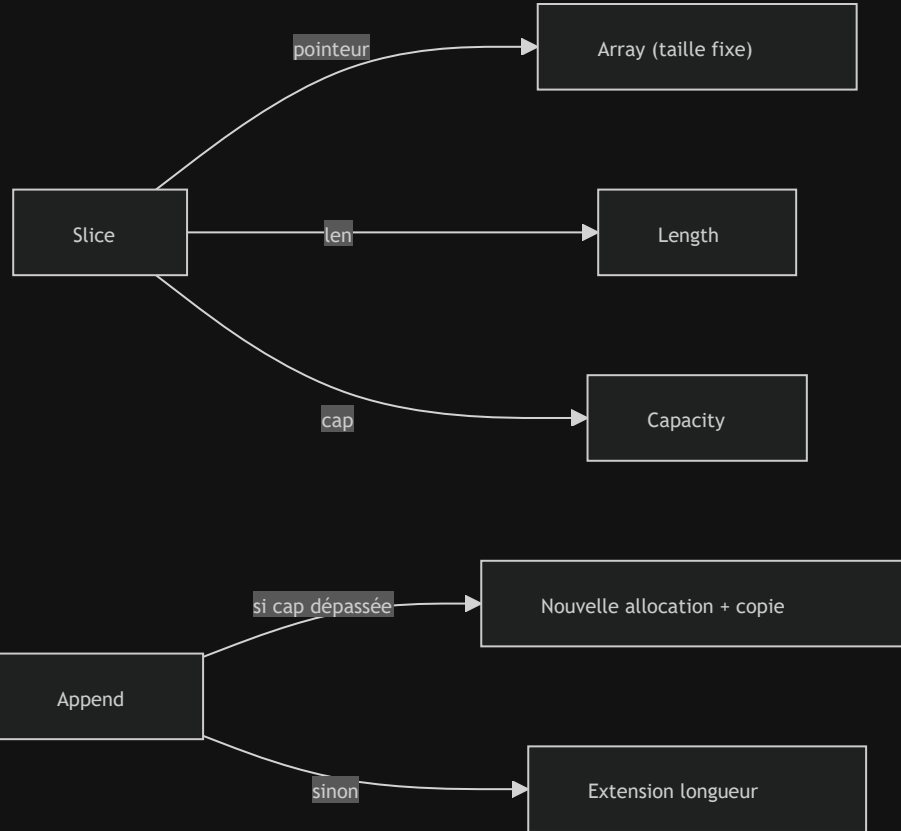
Adopter de bonnes pratiques pour des performances optimales.

- **Optimisations :**
 - **Pré-allouer slices** avec `make` pour éviter trop de copies à chaque `append` .
 - **Minimiser les copies** de slices larges (travailler avec pointeurs quand nécessaire).
 - Utiliser des **types simples en clé de map** pour optimiser performance.
 - **Éviter les réallocations** en prévoyant la taille si possible.

6. Exemples combinés

```
package main
import "fmt"
func main() {
    var arr [3]int = [3]int{10, 20, 30}
    fmt.Println("Array:", arr)
    s := make([]int, 0, 5)
    s = append(s, 1, 2, 3)
    fmt.Printf("Slice: len=%d cap=%d %v\n", len(s), cap(s), s)
    m := make(map[string]int)
    m["alice"] = 25
    m["bob"] = 30
    fmt.Println("Map:", m)
    if age, ok := m["alice"]; ok {
        fmt.Println("Alice a", age, "ans")
    }
}
```

Visualisation des liens entre tableaux et slices.



3-1-1-1

Introduction aux Structs en Go

La Programmation Orientée Structure

- **Go** utilise les **structs** comme fondement de sa programmation orientée structure.
- Les structs sont des **types définis par l'utilisateur** qui regroupent des champs nommés.
- Elles permettent de **modéliser des données composées** (ex: une personne avec un nom et un âge).
- Des **comportements** peuvent leur être associés via des méthodes.
- Ce mécanisme offre une **alternative légère** à la programmation orientée objet classique.

Définition et Instanciation d'une Struct

Qu'est-ce qu'une Struct ?

- Une struct est une **collection ordonnée de champs** de types potentiellement différents.

Syntaxe de définition :

```
type Person struct {  
    Name string // Champ 'Name' de type string  
    Age  int    // Champ 'Age' de type int  
}
```

Instanciation d'une struct :

```
// Création d'une instance de Person  
p := Person{Name: "Alice", Age: 30}
```

3-1-1-3

Composition de Structs (Embedding)

Go : La Composition plutôt que l'Héritage

- Go favorise la **composition** pour réutiliser du code et structurer les données.
- Une struct peut inclure d'autres structs en tant que champs :
 - **Champs nommés** (composition classique).
 - **Champs sans nom (embedding)** : permet d'accéder directement aux champs de la struct embeddée.

Exemple d'Embedding :

```
type Address struct {
    City, Country string
}

type Employee struct {
    Person // Embedding de la struct Person
    Address // Embedding de la struct Address
    ID      int
}

// Accès aux champs emboîtés :
e := Employee{
    Person: Person{Name: "Bob", Age: 40},
    Address: Address{City: "Paris", Country: "France"},
    ID:      123,
}

fmt.Println(e.Name) // Accès direct à Person.Name
fmt.Println(e.City) // Accès direct à Address.City
```

Méthodes Associées aux Structs

Attacher des Comportements aux Types

- Les **méthodes** sont des fonctions associées à un type spécifique (ici, une struct).
- Elles sont définies avec un **receiver**, qui désigne l'instance de la struct sur laquelle la méthode est appelée.

Syntaxe générale d'une méthode :

```
func (p Person) Greet() string { // 'p Person' est le receiver
    return "Bonjour, je m'appelle " + p.Name
}
```

- La méthode `Greet()` est attachée au type `Person`.

Appel de méthode :

```
person := Person{Name: "Alice"}
fmt.Println(person.Greet()) // Affiche : "Bonjour, je m'appelle Alice"
```

Receivers : Valeur vs Pointeur

L'Impact du Receiver sur les Modifications

Le receiver d'une méthode peut être une **valeur** ou un **pointeur**, ce qui détermine comment la struct est manipulée.

- **Receiver en valeur** (`(p Person)`) :

- La méthode travaille sur une **copie** de la struct.
- Les modifications à l'intérieur de la méthode **ne modifient pas l'instance originale**.

```
func (p Person) HaveBirthdayValue() {  
    p.Age += 1 // Modifie la copie uniquement  
}
```

- **Receiver en pointeur** (`(p *Person)`) :

- La méthode travaille sur l'**instance originale** de la struct.
- Les modifications à l'intérieur de la méthode **affectent l'instance d'origine**.

```
func (p *Person) HaveBirthdayPointer() {  
    p.Age += 1 // Modifie l'instance d'origine
```

Exemple d'utilisation :

```
p := Person{Name: "Alice", Age: 29}
p.HaveBirthdayValue() // Age reste 29
// p.HaveBirthdayPointer() // Age devient 30
fmt.Println(p.Age) // 29 si receiver valeur, 30 si pointeur
```

Exemple Complet

Struct, Méthodes et Receivers en Action

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

// Méthode avec receiver pointeur : modifie l'instance originale
func (p *Person) HaveBirthday() {
    p.Age++
}

// Méthode avec receiver valeur : travaille sur une copie
func (p Person) Greet() string {
    return "Bonjour, je m'appelle " + p.Name
}
```



```
func main() {  
    alice := Person{Name: "Alice", Age: 30}  
    fmt.Println(alice.Greet()) // Appel de méthode avec receiver valeur  
    // Affiche : "Bonjour, je m'appelle Alice"  
  
    alice.HaveBirthday() // Appel de méthode avec receiver pointeur  
    fmt.Println("Age après anniversaire :", alice.Age)  
    // Affiche : "Age après anniversaire : 31"  
}
```

3-1-1-8

Conclusion et Références

L'Essence de la Programmation Structurée en Go

- Les **structs**, la **composition par embedding**, et les **méthodes avec receiver** constituent le cœur de la programmation orientée structure en Go.
- Ce modèle favorise un **design simple et puissant**, adapté à la structuration des données et à l'association de comportements.

Sources pour approfondir

- [Go by Example - Structs](#)
- [Go by Example - Methods](#)
- [Effective Go - Methods](#)
- [Go Language Spec - Method declarations](#)
- [Go Wiki - Composition vs Inheritance](#)

3-2-1-1

Introduction aux Interfaces Go

- **Approche minimaliste et puissante** de la programmation orientée structure.
- Go s'appuie sur le **duck typing** et la **satisfaction implicite d'interfaces**.
- La compatibilité d'un type avec une interface se fait **automatiquement** s'il implémente les méthodes requises.
- L'**interface vide** (`interface{}`) représente un type universel capable d'accepter toute valeur.

3-2-1-2

Le Duck Typing en Go

- **Définition :** "If it walks like a duck and quacks like a duck, then it is a duck."
- Un type est compatible avec une interface s'il possède les **méthodes déclarées** dans cette interface.
- **Pas besoin de déclarer explicitement** cette compatibilité.

Duck Typing : Exemple Pratique

```
type Speaker interface {  
    Speak() string  
}  
  
type Dog struct{}  
func (d Dog) Speak() string { return "Woof!" }  
  
type Cat struct{}  
func (c Cat) Speak() string { return "Meow!" }  
  
func makeSpeak(s Speaker) {  
    fmt.Println(s.Speak())  
}  
  
func main() {  
    dog := Dog{}  
    cat := Cat{}  
    makeSpeak(dog) // Woof!  
    makeSpeak(cat) // Meow!  
}
```

- `Dog` et `Cat` sont compatibles avec `Speaker` car ils ont la méthode `Speak()`, sans déclaration explicite.

L'Interface vide (`interface{}`)

- Ne contient **aucune méthode**.
- Est satisfaite par **tous** les types.
- Agit comme un type "universel".

Usages :

- Lorsqu'une fonction doit accepter n'importe quel type.
- Pour stocker des valeurs de types différents dans des slices ou maps.

Exemple :

```
func printValue(v interface{}) {  
    fmt.Println(v)  
}  
  
func main() {  
    printValue(42)  
    printValue("Go")  
    printValue(struct{ Name string }{"Alice"})  
}
```

3-2-1-5

Satisfaction Implicite des Interfaces

- **Pas de mot-clé** `implements` nécessaire en Go.
- Le compilateur vérifie **automatiquement** si un type possède toutes les méthodes d'une interface.
- Préviend la surcharge des annotations et rend la programmation plus flexible.

Contrôle explicite (optionnel) :

- Pour rendre explicite qu'un type implémente une interface, on peut ajouter une assertion compilateur :

```
var _ Speaker = Dog{}
```

- Si `Dog` ne satisfait pas `Speaker`, la compilation échouera.

3-2-1-6

Exemple Combiné : Stringer et interface{}

```
package main

import "fmt"

type Stringer interface {
    String() string
}

type Person struct {
    Name string
    Age  int
}
```



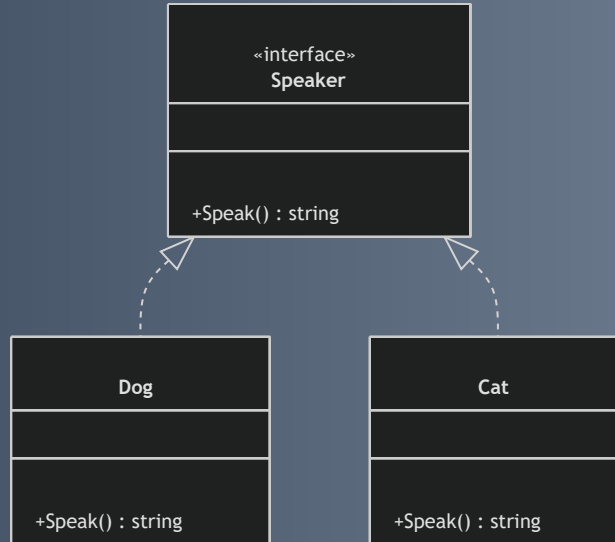
```
func (p Person) String() string {
    return fmt.Sprintf("%s (%d ans)", p.Name, p.Age)
}

func printString(s Stringer) {
    fmt.Println(s.String())
}

func main() {
    p := Person{"Alice", 30}
    printString(p) // Utilisation de l'interface Stringer

    var i interface{} = 10
    fmt.Println(i) // Affiche 10, interface vide acceptant un int
}
```

Diagramme : Relation Type-Interface (Duck Typing)



Ressources & Conclusion

Ressources :

- [Go by Example - Interfaces](#)
- [Go Blog - Interfaces](#)
- [Official Go Specification - Interfaces](#)
- [Effective Go - Interfaces](#)
- [Go Wiki - Interface Satisfaction](#)

Conclusion :

- L'approche des interfaces en Go mélange **simplicité et puissance** via la satisfaction implicite et le duck typing.
- Ouvre des possibilités étendues pour la **programmation modulaire et flexible**.
- L'interface vide permet une **gestion polymorphe** sans contrainte stricte.

3-3-1-1

Introduction à l'Embedding en Go

- **Go ne propose pas d'héritage classique** comme dans d'autres langages orientés objet.
- Il privilégie la **composition** via l'**embedding** pour construire des types complexes.
- Cette approche évite certains problèmes associés à l'héritage.
- Elle offre la **promotion de méthodes**, une forme pratique de délégation.

3-3-1-2

Composition vs Héritage

Héritage (classique)	Composition (Go via embedding)
Relation "est un" (is-a)	Relation "a un" (has-a)
Sous-classe dérivée d'une super-classe	Un type inclut un autre type en champ (embedding)
Héritage des données et comportements	Recyclage des comportements sans création hiérarchique
Peut générer un couplage fort	Plus flexible et découplé

- En Go, un struct peut **embeder** un autre struct.
- Ses champs et méthodes sont automatiquement accessibles comme s'ils appartenaient directement au struct englobant (promotion).

3-3-1-3

Syntaxe de l'Embedding

- Un type est inclus dans un autre **sans nommer explicitement le champ**.
- Le type embarqué devient un **champ anonyme**.

```
type Address struct {  
    City, Country string  
}  
  
type Person struct {  
    Name string  
    Age  int  
    Address // embedding d'Address  
}
```

3-3-1-4

Promotion de Méthodes et Champs

- Les méthodes et champs du type embarqué sont **promus** dans le type englobant.
- On peut y accéder directement.

Exemple :

```
func (a Address) FullAddress() string {
    return a.City + ", " + a.Country
}

p := Person{
    Name: "Alice",
    Age: 29,
    Address: Address{City: "Paris", Country: "France"},
}

fmt.Println(p.FullAddress()) // Accès direct à la méthode Address
```

3-3-1-5

Résolution des Conflits

- Si plusieurs embeddings fournissent des méthodes du même nom, Go exige une **résolution explicite**.

```
type A struct{}
func (A) Hello() { fmt.Println("Hello from A") }

type B struct{}
func (B) Hello() { fmt.Println("Hello from B") }

type C struct {
    A
    B
}

c := C{}
// c.Hello() // Erreur : ambigu car A et B ont Hello
c.A.Hello() // Résolution explicite
c.B.Hello()
```


3-3-1-6

Usages Typiques & Exemple Complet

Usages typiques :

- **Réutilisation de code** : intégrer des comportements communs.
- **“Mixin” léger** : enrichir un struct avec des fonctionnalités.
- **Imiter une hiérarchie** minimaliste sans complexité.

Exemple complet :

```
package main

import "fmt"

type Logger struct{}

func (Logger) Log(msg string) {
    fmt.Println("Log:", msg)
}

type Server struct {
    Address string
    Logger  // embedding de Logger
}

func main() {
    s := Server{Address: "localhost:8080"}
    s.Log("Démarrage du serveur") // Méthode promue du Logger
}
```

3-3-1-8

Conclusion & Ressources

- L'**embedding** en Go offre un mécanisme simple et puissant de composition.
- Il favorise la réutilisation via la promotion de méthodes.
- Il évite la complexité et les pièges liés à l'héritage classique.

Sources :

- [Go by Example - Embedding](#)
- [Effective Go - Embedding](#)
- [Go Blog - Composition using embedding](#)
- [Go Language Specification - Struct types](#)

3-4-1-1

Gestion des erreurs en Go : Les fondamentaux

La gestion des erreurs en Go repose sur des conventions simples mais puissantes, centrées autour du type `error`.

Pour manipuler efficacement les erreurs, Go propose plusieurs outils :

- Le pattern `error` standard.
- Les fonctions `errors.Is` et `errors.As` pour l'inspection et le typage des erreurs.
- La création d'erreurs personnalisées.

Ces mécanismes facilitent un contrôle précis du flux d'erreur dans tout programme, améliorant la robustesse et la maintenabilité du code.

3-4-1-2

Le pattern `error` en Go

Le type `error` est une interface standard définie ainsi :

```
type error interface {  
    Error() string  
}
```

Une fonction retourne souvent un deuxième résultat `error` pour signaler une erreur.

Exemple :

```
func Divide(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, fmt.Errorf("division par zéro")  
    }  
    return a / b, nil  
}  
  
result, err := Divide(10, 0)  
if err != nil {  
    fmt.Println("Erreur:", err) // Affiche "Erreur: division par zéro"  
}
```

- Retourner `nil` dans `error` signifie absence d'erreur.

3-4-1-3

Inspecter les erreurs : `errors.Is`

`errors.Is` permet de vérifier si une erreur imbriquée correspond à une erreur donnée. Il est souvent utilisé avec des erreurs pré-définies ou des valeurs constantes.

Exemple :

```
var ErrNotFound = errors.New("ressource non trouvée")

func find(id int) error {
    // %w permet d'enrouler (wrap) l'erreur
    return fmt.Errorf("find %d: %w", id, ErrNotFound)
}

err := find(10)
if errors.Is(err, ErrNotFound) {
    fmt.Println("Erreur de type NotFound détectée") // Affiche
}
```

La fonction `fmt.Errorf` avec `%w` permet d'enrouler (`wrap`) l'erreur, créant une chaîne d'erreurs.

3-4-1-4

Inspecter les erreurs : `errors.As`

`errors.As` permet de récupérer une erreur spécifique dans une chaîne d'erreurs imbriquées. Il est particulièrement utile avec des erreurs personnalisées structurées.

Exemple :

```
type MyError struct {
    Code int
    Msg  string
}

func (e *MyError) Error() string {
    return e.Msg
}

func mayFail() error {
    return &MyError{Code: 404, Msg: "page non trouvée"}
}

var myErr *MyError
err := mayFail()
if errors.As(err, &myErr) {
    fmt.Println("Code erreur:", myErr.Code) // Affiche "Code erreur: 404"
}
```

`errors.As` tente de "dérouler" la chaîne d'erreurs jusqu'à trouver une erreur du type spécifié et l'assigne à la variable fournie.

3-4-1-5

Création d'erreurs personnalisées

On crée un type qui implémente l'interface `error` pour enrichir les erreurs de contexte, codes ou champs supplémentaires.

Exemple :

```
type ValidationError struct {
    Field    string
    Message  string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("Champ %s invalide: %s", e.Field, e.Message)
}

func validate(name string) error {
    if name == "" {
        return &ValidationError{Field: "Name", Message: "ne peut pas être vide"}
    }
    return nil
}

err := validate("")
if ve, ok := err.(*ValidationError); ok {
    fmt.Println("Erreur de validation:", ve) // Affiche "Erreur de validation: Champ Name invalide: ne peut pas être vide"
}
```

Cela permet de transporter des informations spécifiques à l'erreur au-delà du simple message.

3-4-1-6

Exemple pratique : Combiner les outils

```
package main

import (
    "errors"
    "fmt"
)

var ErrNotFound = errors.New("ressource non trouvée")

type ValidationError struct {
    Field string
    Msg    string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("Champ %s invalide: %s", e.Field, e.Msg)
}

→Suite→
```

```
func findResource(id int) error {
    return fmt.Errorf("find %d: %w", id, ErrNotFound)
}

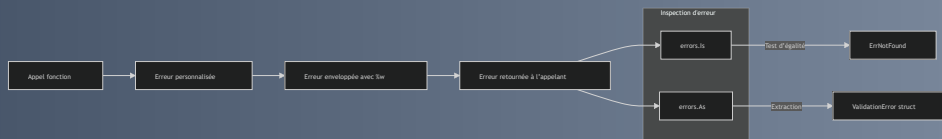
func validateName(name string) error {
    if name == "" {
        return &ValidationError{Field: "Name", Msg: "ne peut pas être vide"}
    }
    return nil
}

func main() {
    err := findResource(42)
    if errors.Is(err, ErrNotFound) {
        fmt.Println("Erreur trouvée:", err) // Affiche "Erreur trouvée: find 42: ressource non trouvée"
    }

    err = validateName("")
    var ve *ValidationError
    if errors.As(err, &ve) {
        fmt.Println("ValidationError détectée:", ve) // Affiche "ValidationError détectée: Champ Name invalide: ne peut
    }
}
```

3-4-1-7

Visualiser la chaîne d'erreurs et l'inspection



Ce diagramme illustre comment une erreur peut être enveloppée et comment `errors.Is` et `errors.As` permettent d'inspecter cette chaîne pour des types ou des valeurs spécifiques.

3-4-1-8

Résumé et pour aller plus loin

En résumé, Go propose un pattern `error` simple et efficace, enrichi avec `errors.Is` et `errors.As` pour une inspection fine. La création d'erreurs personnalisées facilite la capture contextuelle et détaillée des erreurs. Utiliser ces outils améliore la robustesse et la maintenabilité du code.

Sources :

- [Go Blog - Error values](#)
- [Pkg errors - errors.Is, errors.As](#)
- [Go by Example - Errors](#)
- [Effective Go - Errors](#)
- [Go Wiki - Error handling and inspection](#)

3-5-1-1

Paradigme Go : Comportements et Interfaces

L'absence de classes pour une programmation flexible

- Go adopte un paradigme structuré autour des **comportements**.
- Pas de classes ni d'héritage au sens classique.
- La clé réside dans les **interfaces** et la **composition de types** via les **structs**.
- Favorise la flexibilité, la simplicité et un code fortement découplé.

3-5-1-2

Go n'a pas de classes : Structs et Interfaces

- Contrairement aux langages orientés objet classiques (Java, C++, C#), Go ne propose **pas de classes**.
- Il utilise :
 - Des **structs** pour modéliser des données composites.
 - Des **interfaces** pour décrire des comportements.
- Les structs ne contiennent que des données, sans notion d'héritage.

3-5-1-3

Penser en termes de comportements : Les Interfaces

- Une **interface** décrit un ensemble de méthodes qu'un type doit implémenter.
- Un type satisfait **implicitement** une interface simplement en implémentant ses méthodes, sans déclaration explicite.

Exemple d'interface simple :

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}
```

- Tout type possédant une méthode `Read([]byte) (int, error)` satisfait automatiquement cette interface.

L'Interface comme contrat comportemental

- En Go, on ne pense pas "qu'est-ce que c'est ?" mais "**qu'est-ce que ça peut faire ?**".
 - Exemple : un type peut "démarrer", "accélérer", "freiner" (comportements via interfaces).
- Cela facilite la programmation modulaire et le polymorphisme.

Exemple de polymorphisme :

```
func PlaySound(p Player) {  
    p.Play()  
}  
  
type Player interface {  
    Play()  
}  
  
type MP3Player struct{}  
func (m MP3Player) Play() { fmt.Println("Lecture MP3") }  
  
type CDPlayer struct{}  
func (c CDPlayer) Play() { fmt.Println("Lecture CD") }
```

3-5-1-5

Composition et Réutilisation : L'Alternative à l'Héritage

- Au lieu d'héritage, Go utilise la **composition**.
- Elle s'effectue via l'**embedding de structs**.
- Permet de construire des types complexes en combinant d'autres types et leurs méthodes.

3-5-1-6

Exemple complet : Interfaces en action

```
package main

import "fmt"

type Speaker interface {
    Speak() string
}

type Dog struct{}

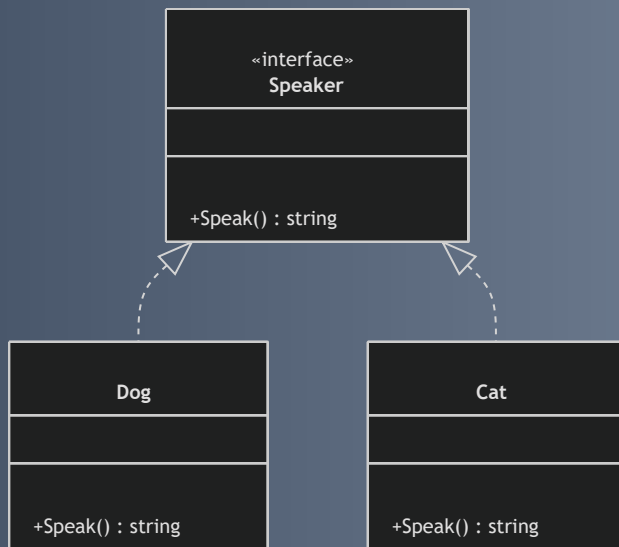
func (d Dog) Speak() string {
    return "Woof"
}

type Cat struct{}
```

```
func (c Cat) Speak() string {  
    return "Meow"  
}  
  
func makeSpeak(s Speaker) {  
    fmt.Println(s.Speak())  
}  
  
func main() {  
    var s Speaker  
  
    s = Dog{}  
    makeSpeak(s) // Affiche "Woof"  
  
    s = Cat{}  
    makeSpeak(s) // Affiche "Meow"  
}
```

3-5-1-7

Visualisation : Paradigme Go orienté comportement



3-5-1-8

Synthèse et Ressources

- **Synthèse** : Go incite à concevoir les programmes comme un ensemble de types décrivant **ce qu'ils font**, non ce qu'ils sont.
- Élimine les contraintes de l'héritage classique tout en conservant puissance et expressivité via interfaces et composition.

Sources :

- [Go by Example - Interfaces](#)
- [Effective Go - Interfaces](#)
- [Go Blog - Interfaces](#)
- [Go Language Specification - Interface types](#)

4-1-1-1

Concurrence en Go : Goroutines et Scheduler

- **Modèle de Concurrence Efficace**
 - Go utilise les **goroutines**, des unités d'exécution légères.
 - Elles sont multiplexées par un **scheduler intégré**.
- **Modèle M:N**
 - Contrairement aux threads système, Go mappe **M goroutines** sur un nombre limité de **N threads système**.
 - Ceci optimise l'utilisation des ressources.

Créer une Goroutine : Simple et Efficace

- **Syntaxe Simple**

- Ajoutez le mot-clé `go` devant un appel de fonction.
- La fonction s'exécute de manière concurrente, en arrière-plan.

- **Exemple Minimal**

```
func sayHello() {  
    fmt.Println("Hello from goroutine")  
}  
  
func main() {  
    go sayHello() // Lancement d'une goroutine  
    time.Sleep(time.Second) // Attendre pour observer la sortie  
}
```

- **Coût Réduit**

- Très rapide à créer (quelques kilo-octets de pile initiale).
- Peu coûteuse en mémoire, à la différence d'un thread système.

4-1-1-3

Le Cycle de Vie d'une Goroutine

- **Création**
 - Via `go func() { ... }()`
- **Exécution**
 - Planifiée par le scheduler Go.
- **Blocage**
 - Si la goroutine attend une ressource, un canal, ou via un appel bloquant.
- **Extinction**
 - Lorsqu'elle termine l'exécution de sa fonction.
- **Gestion de la Pile**
 - Go gère dynamiquement la pile mémoire des goroutines, qui peut grandir ou diminuer.

Le Scheduler Go : Comprendre le Modèle M:N

- **Multiplexage M:N**

- Le scheduler Go multiplexe **M goroutines** sur **N threads système**.

- **Composants Clés**

- **G (goroutine)** : Unité d'exécution logique.
- **M (machine OS)** : Thread système réel.
- **P (processor)** : Ressource logique d'exécution qui possède un contexte pour exécuter des G sur un M.

- **Fonctionnement**

- Plusieurs goroutines sont prêtes et placées dans une queue par P.
- Chaque P est associé à un thread OS (M) qui exécute les goroutines.
- Si une goroutine est bloquée (ex: IO), Go déplace une autre goroutine prête pour une exécution efficace.

4-1-1-5

Caractéristiques Clés : Performance et Scalabilité

- **Empilement Dynamique**
 - La pile commence à une petite taille (~2 KB).
 - Elle s'agrandit ou se réduit automatiquement selon les besoins.
- **Équilibrage de Charge**
 - Le scheduler répartit le travail entre les P et M de façon concurrente.
- **Prise en Charge du Parallélisme**
 - Le runtime permet d'augmenter le nombre de P (via `runtime.GOMAXPROCS`) pour utiliser plusieurs cœurs de processeur.

4-1-1-6

Goroutines en Action : Exemple Pratique

```
package main

import (
    "fmt"
    "runtime"
    "sync"
)

func worker(id int, wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Printf("Worker %d starting\n", id)
    for i := 0; i < 3; i++ {
        fmt.Printf("Worker %d working %d\n", id, i)
        runtime.Gosched() // yield pour permettre au scheduler de switcher
    }
    fmt.Printf("Worker %d done\n", id)
}
```

```
func main() {  
    runtime.GOMAXPROCS(2) // utiliser 2 threads OS  
  
    var wg sync.WaitGroup  
  
    for i := 1; i ≤ 3; i++ {  
        wg.Add(1)  
        go worker(i, &wg)  
    }  
    wg.Wait()  
}
```

Visualisation : Le Scheduler Go en Action

4-1-1-8

Conclusion et Pour Aller Plus Loin

- **Conclusion**

- Le modèle M:N de Go, avec ses goroutines légères et son scheduler intégré, gère efficacement la concurrence.
- Il optimise l'utilisation des threads système sans surcharge importante, garantissant simplicité et scalabilité.

- **Ressources**

- [Go Blog - Goroutines](#)
- [Go Blog - The Go Scheduler](#)
- [Go source code - runtime](#)
- [Effective Go - Concurrency](#)
- [Go by Example - Goroutines](#)

4-2-1-1

Introduction aux Channels en Go

- Les **channels** sont un mécanisme fondamental en Go pour la **synchronisation** et la **communication** entre **goroutines**.
- Ils permettent l'échange sécurisé et ordonné de données, à travers un concept proche des "pipes".
- Go distingue plusieurs types de channels selon leur capacité (bufferisé ou non) et leur direction (lecture, écriture).

4-2-1-2

Channels Non Bufferisés (Unbuffered)

- Un **channel non bufferisé** est un canal où chaque envoi (`chan ←`) attend qu'un récepteur (`← chan`) soit prêt à recevoir la donnée, et vice versa.
- Cela assure une **synchronisation stricte** entre l'émetteur et le récepteur.

```
ch := make(chan int) // channel non bufferisé

go func() {
    ch ← 42 // envoi bloquant jusqu'à ce que quelqu'un lise
}()

val := ←ch // réception (bloquante si rien n'est envoyé)
fmt.Println(val) // 42
```

* Cet échange bloquant garantit que la donnée est transférée de façon synchrone.

4-2-1-3

Channels Bufferisés (Buffered)

- Un **channel bufferisé** possède une capacité N, ce qui signifie qu'il peut stocker N éléments sans que l'émetteur soit bloqué, tant que le buffer n'est pas plein.
- Le récepteur peut lire plus tard.

```
ch := make(chan int, 2) // channel bufferisé de capacité 2

ch ← 1 // non bloquant (buffer vide)
ch ← 2 // non bloquant (buffer à moitié plein)

val1 := ←ch
val2 := ←ch
fmt.Println(val1, val2) // 1 2
```

* L'usage du buffer améliore la flexibilité et les performances, en réduisant la nécessité de synchronisation stricte.

4-2-1-4

Channels Directionnels

- Go permet de restreindre un channel à une **direction unique** afin de clarifier l'intention du code :
 - `chan← T` : channel en **écriture seule**
 - `←chan T` : channel en **lecture seule**
- Cela renforce la sécurité et la lisibilité.

```
func send(ch chan← int, val int) {  
    ch ← val // ne peut qu'écrire  
}  
  
func receive(ch ←chan int) int {  
    return ←ch // ne peut que lire  
}
```

- Ce typage réduit les erreurs accidentelles de lecture ou écriture.

4-2-1-5

Usages Typiques des Channels

- **Synchronisation** : garantir qu'une goroutine attend une autre (via un channel non bufferisé).
- **Communication** : transmettre des données entre goroutines.
- **Pipeline** : chaîner plusieurs étapes de traitement via des channels.

4-2-1-6

Exemple Complet : Producteur-Consommateur

```
package main

import (
    "fmt"
    "time"
)

func producer(ch chan<- int) {
    for i := 1; i ≤ 3; i++ {
        fmt.Printf("Producing %d\n", i)
        ch ← i
        time.Sleep(300 * time.Millisecond)
    }
    close(ch) // Important : signale la fin de production
}
```

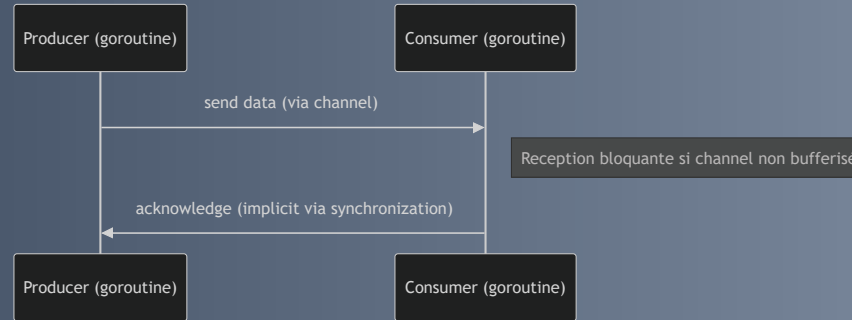
```
func consumer(ch <-chan int) {  
    for val := range ch { // Boucle tant que le channel est ouvert et non vide  
        fmt.Printf("Consumed %d\n", val)  
    }  
}  
  
func main() {  
    ch := make(chan int, 2) // buffered channel  
  
    go producer(ch)  
    go consumer(ch)  
}
```

****Sortie :****

```
Producing 1  
Consumed 1  
Producing 2  
Consumed 2  
Producing 3  
Consumed 3
```


4-2-1-7

Diagramme de Communication via Channels



4-2-1-8

Conclusion : L'Essentiel sur les Channels

- Les channels constituent un outil de communication et synchronisation **élégant** assurant l'échange contrôlé d'informations entre goroutines.
- La distinction entre channels **bufferisés** et **non bufferisés** permet d'adapter le modèle pour obtenir la concurrence la plus efficace selon les besoins.
- Le **typage directionnel** renforce la robustesse du code.

4-3-1-1

Les patterns

Go facilite la construction de systèmes concurrents robustes grâce à plusieurs **patterns** classiques. Ces patterns utilisent les outils natifs comme les goroutines et les channels.

Parmi ces patterns essentiels figurent :

- **Fan-out**
- **Fan-in**
- **Pipeline**
- **Worker Pool**

Ils permettent de structurer efficacement le traitement concurrent et la synchronisation des tâches.

1. Fan-out : Paralléliser les tâches

```
func worker(id int, jobs ←chan int, results chan← int) {  
    for j := range jobs {  
        fmt.Printf("Worker %d traite job %d\n", id, j)  
        results ← j * 2  
    }  
}
```

```
func main() {  
    jobs := make(chan int, 5)  
    results := make(chan int, 5)  
  
    // Démarrage de 3 workers en fan-out  
    for w := 1; w ≤ 3; w++ {  
        go worker(w, jobs, results)  
    }  
  
    // Envoi des jobs  
    for j := 1; j ≤ 5; j++ {  
        jobs ← j  
    }  
    close(jobs)  
  
    // Récupération des résultats  
    for a := 1; a ≤ 5; a++ {  
        fmt.Println("Résultat:", ←results)  
    }  
}
```

2. Fan-in : Agréger les résultats

Le **fan-in** consiste à agréger les sorties de plusieurs goroutines dans un seul channel, facilitant ainsi la collecte et la gestion des résultats.

```
func fanIn(cs ...←chan int) ←chan int {  
    out := make(chan int)  
    var wg sync.WaitGroup  
    wg.Add(len(cs))  
  
    for _, c := range cs {  
        go func(ch ←chan int) {  
            defer wg.Done()  
            for v := range ch {  
                out ← v  
            }  
        }(c)  
    }  
  
    go func() {  
        wg.Wait()  
        close(out)  
    }()  
    return out  
}
```

3. Pipeline : Chaîne de traitement par étapes

Le **pipeline** crée une chaîne de traitement par étapes, chaque étape exécutée dans une goroutine recevant et envoyant des données sur un channel.

```
func gen(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        for _, n := range nums {
            out <- n
        }
        close(out)
    }()
    return out
}

func sq(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        for n := range in {
            out <- n * n
        }
        close(out)
    }()
    return out
}
```

```
func main() {  
    nums := gen(2, 3, 4)  
    squares := sq(nums)  
  
    for n := range squares {  
        fmt.Println(n)  
    }  
}
```

4-3-1-5

4. Worker Pool : Gestion optimisée des tâches

Le **worker pool** est une forme spécialisée de fan-out où un nombre fixe de goroutines partagent la charge des jobs envoyés sur un channel, évitant la surcharge des ressources.

Exemple :

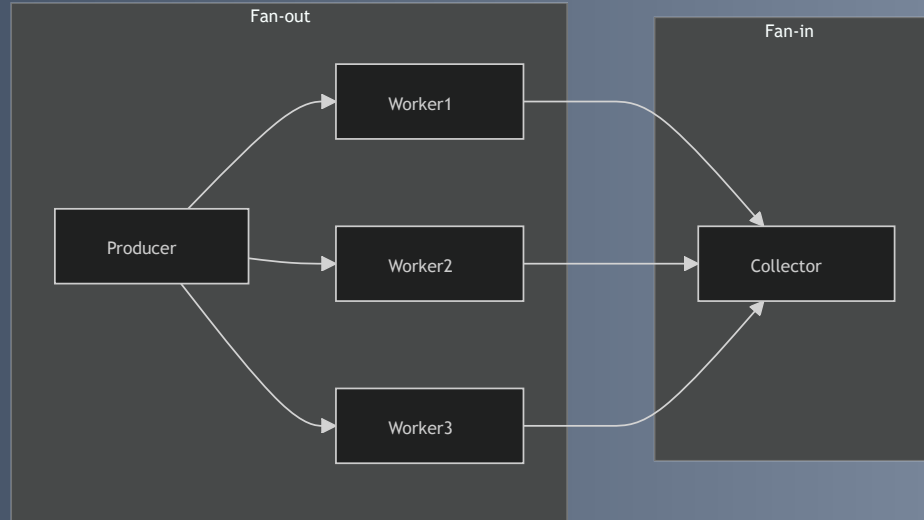
```
func worker(id int, jobs ←chan int, results chan← int) {
    for j := range jobs {
        fmt.Printf("Worker %d démarrage job %d\n", id, j)
        time.Sleep(time.Second) // simuler travail
        results ← j * 2
        fmt.Printf("Worker %d terminé job %d\n", id, j)
    }
}
```



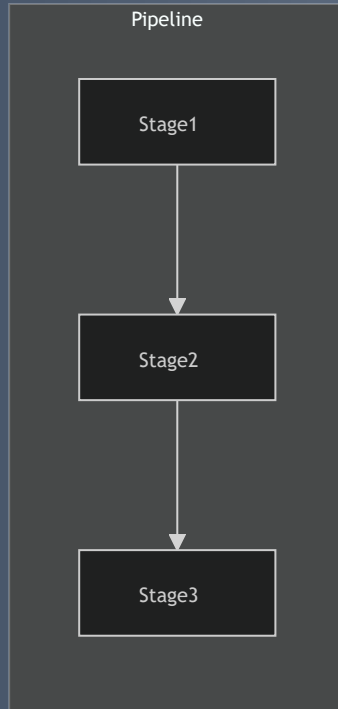
```
func main() {  
    const numJobs = 5  
    jobs := make(chan int, numJobs)  
    results := make(chan int, numJobs)  
  
    // Démarrage de 3 workers  
    for w := 1; w ≤ 3; w++ {  
        go worker(w, jobs, results)  
    }  
  
    // Envoi des jobs  
    for j := 1; j ≤ numJobs; j++ {  
        jobs ← j  
    }  
    close(jobs)  
  
    // Collecte des résultats  
    for a := 1; a ≤ numJobs; a++ {  
        fmt.Println("Résultat:", ←results)  
    }  
}
```

4-3-1-6

5. Vue globale des patterns



5. Vue globale des patterns



5. Vue globale des patterns



4-3-1-7

6. Sources pour approfondir

- [Go Blog - Go Concurrency Patterns: Pipelines and cancellations](#)
- [Go by Example - Worker Pools](#)
- [Effective Go - Concurrency Patterns](#)
- [Go Wiki - Concurrency Patterns](#)

Conclusion : Bâtir des applications concurrentes robustes

Ces patterns – fan-out, fan-in, pipeline et worker pool – fournissent des bases robustes pour organiser la concurrence en Go.

Leur compréhension permet de bâtir des applications concurrentes :

- Performantes
- Simples à maintenir
- Faciles à faire évoluer

4-4-1-1

Select en Go – Multiplexage de channels pour gérer la concurrence

Introduction au `select`

- Le mot-clé `select` est un outil natif de Go.
- Il permet de **multiplexer des opérations sur plusieurs channels**.
- Une goroutine peut attendre simultanément sur plusieurs communications.
- Réagit dès qu'une des opérations devient disponible.
- **Objectif** : Simplifier la gestion concurrente, la synchronisation et la coordination entre goroutines.

Exemple simple d'attente sur plusieurs canaux - Gérer les messages selon leur disponibilité

```
func main() {  
    ch1 := make(chan string)  
    ch2 := make(chan string)  
  
    go func() {  
        time.Sleep(1 * time.Second)  
        ch1 ← "message de ch1"  
    }()  
    go func() {  
        time.Sleep(2 * time.Second)  
        ch2 ← "message de ch2"  
    }()  
  
    for i := 0; i < 2; i++ {  
        select {  
        case msg := ←ch1:  
            fmt.Println("Reçu :", msg)  
        case msg := ←ch2:  
            fmt.Println("Reçu :", msg)  
        }  
    }  
}
```

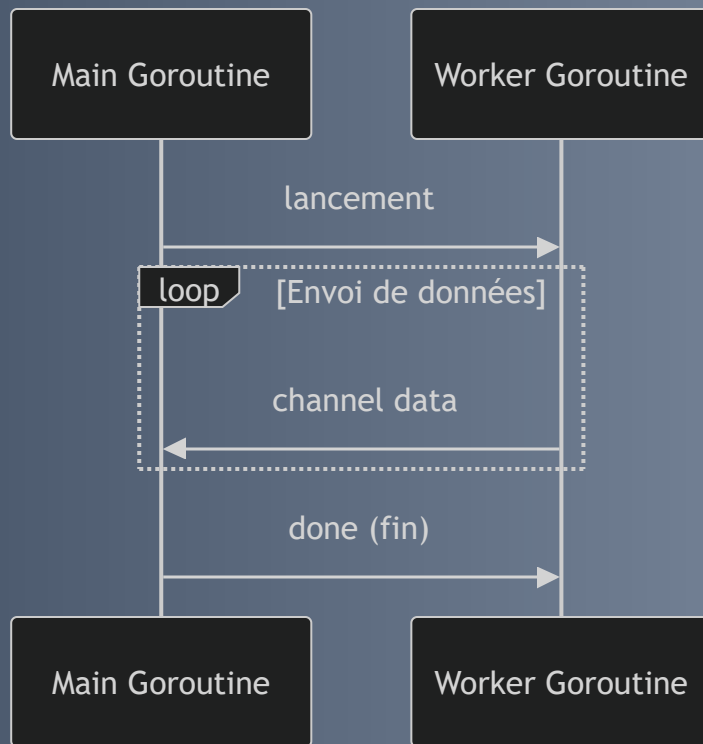
Ce code attend, à tour de rôle, les messages des deux channels en gérant les arrivées selon leur disponibilité.

4-4-1-5

Exemple avec timeout - Éviter le blocage indéfini

```
select {  
  case res := <-resultChan:  
    fmt.Println("Résultat reçu:", res)  
  case <-time.After(500 * time.Millisecond):  
    fmt.Println("Timeout atteint")  
}
```

Cette construction évite de bloquer indéfiniment une goroutine en attendant un résultat.

Flux avec `select`

4-4-1-8

Conclusion et Ressources

`select` : un mécanisme puissant pour la concurrence en Go

- `select` offre un mécanisme concis, puissant et flexible.
- Il gère la concurrence et le multiplexage d'opérations sur channels.
- Facilite l'écriture de programmes concurrentiels fiables et responsives.

Sources

- [Go Blog - Go's select statement](#)
- [Go by Example - Select](#)
- [The Go Programming Language Specification - Select statements](#)
- [Effective Go - Concurrency](#)

Synchronisation bas niveau en Go - Introduction

- **La Concurrency en Go**

- Go excelle dans la gestion de la concurrence grâce aux **goroutines**.
- Mais la concurrence nécessite de la **synchronisation** pour éviter les problèmes.

- **Pourquoi la Synchronisation ?**

- Contrôler l'accès aux ressources partagées.
- Coordonner le comportement des goroutines.

- **Primitives Essentielles du Package `sync`**

- `sync.Mutex` : Pour la protection des données.
- `sync.WaitGroup` : Pour la coordination des tâches.
- `sync.Once` : Pour l'initialisation unique.

4-5-1-2

`sync.Mutex` - Verrou d'exclusion mutuelle

- **Rôle**

- Protège une ressource partagée contre les accès concurrents non contrôlés.
- Garantit qu'une seule goroutine accède à une section critique à la fois.

- **Fonctionnement**

- `Lock()` : Verrouille le Mutex. Si le Mutex est déjà verrouillé, la goroutine appelante est bloquée jusqu'à ce qu'il soit déverrouillé.
- `Unlock()` : Déverrouille le Mutex, permettant à d'autres goroutines d'acquérir le verrou.

- **Sans Mutex**

- Des **conditions de course** peuvent corrompre la valeur des données partagées.

4-5-1-4

`sync.WaitGroup` - Attendre un groupe de goroutines

- **Rôle**

- Synchronise une goroutine principale avec un ensemble de goroutines concurrentes.
- Permet à la goroutine principale d'attendre la fin de toutes les goroutines lancées.

- **Fonctionnement**

- `Add(n int)` : Incrmente le compteur interne du `WaitGroup` de `n`. Indique le nombre de goroutines à attendre.
- `Done()` : Décrmente le compteur du `WaitGroup` de 1. Souvent appelé avec `defer` pour garantir son exécution.
- `Wait()` : Bloque la goroutine appelante jusqu'à ce que le compteur du `WaitGroup` atteigne zéro.

sync.WaitGroup - Exemple pratique

```
var wg sync.WaitGroup // Notre WaitGroup

func worker(id int) {
    defer wg.Done() // Indique que cette goroutine est terminée
    fmt.Printf("Worker %d starting\n", id)
    // simulate work
    time.Sleep(time.Second)
    fmt.Printf("Worker %d done\n", id)
}

func main() {
    for i := 1; i ≤ 3; i++ {
        wg.Add(1) // Incrémente le compteur pour chaque worker
        go worker(i)
    }
    wg.Wait() // Attend la fin des 3 workers
    fmt.Println("All workers finished!")
}
```

- La fonction `main` ne se termine qu'après l'exécution de tous les `worker`.

4-5-1-6

`sync.Once` - Exécuter une action une seule fois

- **Rôle**

- Garantit qu'une fonction donnée ne s'exécute qu'une seule fois, quel que soit le nombre d'appels concurrents.
- Idéal pour les initialisations coûteuses ou non idempotentes.

- **Fonctionnement**

- `Do(f func())` : Exécute la fonction `f` une seule fois. Les appels ultérieurs à `Do` avec la même instance de `Once` n'auront aucun effet.

- **Exemple**

```
package main

import (
    "fmt"
    "sync"
    "time" // Nécessaire pour time.Sleep
)
```



```
var once sync.Once // Notre Once

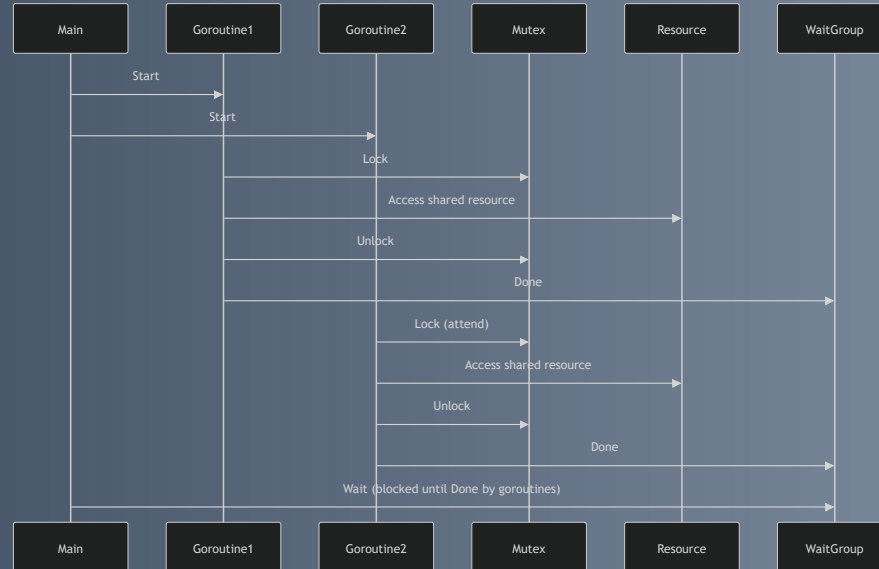
func initialize() {
    fmt.Println("Initialisation unique")
}

func main() {
    for i := 0; i < 5; i++ {
        go func() {
            once.Do(initialize) // Appelle initialize une seule fois
        }()
    }
    time.Sleep(time.Second) // Laisse le temps aux goroutines de s'exécuter
}
```

- `initialize()` sera appelée une seule fois, même si 5 goroutines tentent de l'exécuter.

4-5-1-7

Synchronisation bas niveau - Diagramme



- Ce diagramme illustre comment **Mutex** protège la ressource et **WaitGroup** coordonne la fin des goroutines.

4-5-1-8

Conclusion et Ressources

- **Conclusion**

- Les primitives `sync.Mutex`, `sync.WaitGroup` et `sync.Once` offrent un contrôle précis et sûr.
- Elles sont essentielles pour gérer les interactions concurrentes, éviter la corruption d'état et garantir des exécutions ordonnées dans vos programmes Go.

- **Sources**

- [Package sync - Go documentation](#)
- [Go Blog - Go Concurrency Patterns](#)
- [Go by Example - Mutex](#)
- [Go by Example - WaitGroups](#)
- [Go by Example - Once](#)

Context en Go : Gérer l'annulation et les timeouts

Introduction Le package `context` de Go est conçu pour :

- Gérer l'annulation, les timeouts, et la propagation de signaux entre goroutines et services.
- Faciliter l'écriture de programmes concurrentiels réactifs et contrôlés, notamment en environnement distribué ou dans la gestion des appels réseau.

Le rôle du `context.Context` Un contexte (`context.Context`) encapsule des données de contrôle, telles que :

- **Annulation** (cancel)
- **Timeout**
- **Valeurs associées** (pour passage de métadonnées)
- **Propagation** dans les chaînes d'appels

Il est immuable et toujours passé explicitement en premier paramètre d'une fonction.

4-6-1-2

Création et Annulation Manuelle

`context.WithCancel` permet une annulation explicite.

```
ctx, cancel := context.WithCancel(context.Background())

go func() {
    time.Sleep(2 * time.Second)
    cancel() // annulation déclenchée
}()

select {
case ←time.After(5 * time.Second):
    fmt.Println("Terminé sans annulation")
case ←ctx.Done():
    fmt.Println("Annulé:", ctx.Err()) // ctx.Err() renvoie context.Canceled
}
```

4-6-1-3

Timeout Intégré avec `context.WithTimeout`

`context.WithTimeout` crée un contexte qui s'annule automatiquement après un délai.

```
ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
defer cancel()

select {
case ←time.After(5 * time.Second):
    fmt.Println("Travail terminé")
case ←ctx.Done():
    fmt.Println("Timeout :", ctx.Err()) // context.DeadlineExceeded
}
```

4-6-1-4

Propagation du Contexte dans les Appels

Les fonctions et services doivent recevoir le contexte pour propager annulation et deadlines.

```
func doWork(ctx context.Context) {
    select {
    case <-ctx.Done():
        fmt.Println("Travail annulé:", ctx.Err())
        return
    case <-time.After(10 * time.Second):
        fmt.Println("Travail terminé")
    }
}

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    go doWork(ctx)
    time.Sleep(6 * time.Second)
}
```

4-6-1-5

Passage de Valeurs avec `context.WithValue`

Utilisez `context.WithValue` pour injecter des métadonnées, qui doivent être immuables et uniquement pour des données transversales.

```
type key string

func main() {
    ctx := context.WithValue(context.Background(), key("userID"), "12345")

    process(ctx)
}

func process(ctx context.Context) {
    userID := ctx.Value(key("userID")).(string)
    fmt.Println("User ID:", userID)
}
```


4-6-1-7

Pourquoi le package `context` est indispensable

Le package `context` est indispensable pour écrire des applications Go qui respectent les contraintes de temps, d'annulation et de propagation d'état, particulièrement dans les architectures modernes distribuées et concurrentes. Son usage explicite garantit un contrôle fin du cycle de vie des goroutines et des appels.

4-6-1-8

Sources

- [Package context - documentation officielle](#)
- [Go Blog - Context](#)
- [Go by Example - Context](#)
- [Effective Go - Concurrency](#)

5-1-1-1

Développement d'API REST en Go

Introduction

- Go offre la bibliothèque standard `net/http`, robuste et performante.
- Idéale pour construire des **API REST** de toute complexité.
- Maîtriser les concepts de **handlers**, **routage** et **middleware** est fondamental pour des services backend efficaces.

Créer un serveur HTTP simple avec `net/http`

Le package `net/http`

- Contient les fonctions clés pour écouter sur un port, gérer les requêtes et envoyer des réponses.

```
package main

import (
    "fmt"
    "net/http"
)

func helloHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Bonjour, API REST en Go !")
}

func main() {
    http.HandleFunc("/", helloHandler)
    http.ListenAndServe(":8080", nil)
}
```

- Ce serveur répond "Bonjour, API REST en Go !" à toute requête sur la racine `/`.

5-1-1-3

Handlers et `HandlerFunc`

Qu'est-ce qu'une Handler ?

- En Go, une **handler** est un type qui implémente l'interface `http.Handler` :

```
type Handler interface {  
    ServeHTTP(ResponseWriter, *Request)  
}
```

- La fonction `http.HandleFunc` permet d'utiliser une fonction simple (ayant la signature `func(w http.ResponseWriter, r *http.Request)`) comme handler, grâce à un adaptateur interne.

5-1-1-4

Routage simple avec `http.ServeMux` Le Multiplexeur (`ServeMux`)

- Permet de router différentes URL vers des handlers spécifiques.
- Il agit comme un "aiguilleur" de requêtes.

```
mux := http.NewServeMux()  
mux.HandleFunc("/users", usersHandler)  
mux.HandleFunc("/products", productsHandler)  
http.ListenAndServe(":8080", mux)
```

- Chaque chemin (`/users` , `/products`) est associé à sa propre fonction handler.

5-1-1-5

Middleware en Go

Qu'est-ce qu'un Middleware ?

- Une fonction qui **enveloppe un handler** pour ajouter un comportement transversal.
- Exemples : logging, authentification, gestion des erreurs, CORS.

Exemple de middleware de logging :

```
func loggingMiddleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        fmt.Printf("Requête %s %s\n", r.Method, r.URL.Path)  
        next.ServeHTTP(w, r) // Appelle le handler suivant  
    })  
}
```

```
func main() {  
    mux := http.NewServeMux()  
    mux.HandleFunc("/hello", helloHandler)  
  
    loggedMux := loggingMiddleware(mux) // Applique le middleware au routeur  
  
    http.ListenAndServe(":8080", loggedMux)  
}
```


Exemple complet : API REST simple

Intégration des concepts

```
package main

import (
    "encoding/json"
    "log"
    "net/http"
)

type User struct {
    ID int `json:"id"`
    Name string `json:"name"`
}

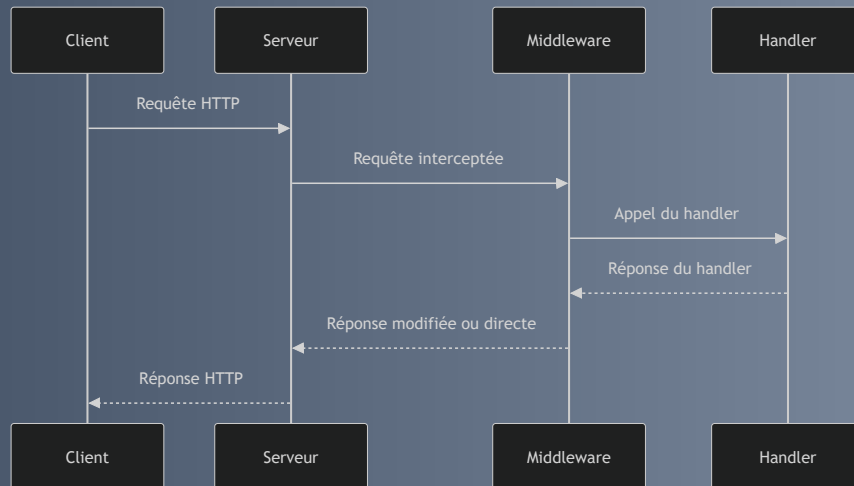
var users = []User{{1, "Alice"}, {2, "Bob"}}

func usersHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "Method Not Allowed", http.StatusMethodNotAllowed)
        return
    }
    w.Header().Set("Content-Type", "application/json")
}
```

```
func loggingMiddleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        log.Printf("%s %s\n", r.Method, r.URL.Path)  
        next.ServeHTTP(w, r)  
    })  
}  
  
func main() {  
    mux := http.NewServeMux()  
    mux.HandleFunc("/users", usersHandler)  
  
    loggedMux := loggingMiddleware(mux) // Le middleware enveloppe le routeur  
  
    log.Println("Serveur démarré sur :8080")  
    http.ListenAndServe(":8080", loggedMux)  
}
```

5-1-1-7

Architecture simplifiée d'une requête HTTP avec middleware



5-1-1-8

Conclusion et Ressources

En résumé

- La création d'API REST en Go repose sur la compréhension de `net/http`.
- Composition claire entre **handlers**, **routage** et **middleware**.
- Cette base permet d'étendre facilement les fonctionnalités, tout en gardant une approche modulaire et maintenable.

Sources

- [Go by Example - HTTP Servers](#)
- [Package net/http - Documentation officielle](#)
- [Go Blog - Writing web applications](#)
- [Go Middleware Patterns](#)

Introduction au framework Gin pour simplifier le développement d'API en Go

Développement backend et exposition de services – Framework Gin

Gin est un framework web léger et performant pour Go, spécialement conçu pour faciliter et accélérer le développement d'API REST.

Il fournit :

- Une API simple
- Un routage rapide
- Un middleware flexible
- Des fonctionnalités utiles : gestion des requêtes JSON, validation, gestion des erreurs.

Le tout en conservant la simplicité du package standard `net/http`.

5-2-1-2

1. Pourquoi choisir Gin ?

Gin est un choix privilégié pour le développement d'API en Go grâce à ses atouts majeurs :

- **Performance élevée** : Parmi les frameworks les plus rapides en Go.
- **Routing expressif** : Supporte les routes avec paramètres, groupes et routes imbriquées.
- **Middleware efficace** : Possibilité d'intercaler des middlewares à tous les niveaux (global, groupe, route).
- **Support JSON natif** : Encodage/décodage facile des données.
- **Gestion intégrée** : Erreurs, validation, récupération des paniques.
- **Documentation et communauté actives**.

5-2-1-3

2. Installation & 3. Premier exemple

```
go get -u github.com/gin-gonic/gin
```

Premier exemple : Un serveur simple

```
package main

import (
    "github.com/gin-gonic/gin"
    "net/http"
)

func main() {
    r := gin.Default() // Crée un routeur avec middleware Logger et Recovery

    r.GET("/ping", func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{"message": "pong"})
    })

    r.Run(":8080") // Démarre le serveur sur le port 8080
}
```

5-2-1-4

4. Routage et Handlers

Gin simplifie l'extraction des paramètres et l'affectation des handlers :

Paramètres d'URL

```
r.GET("/user/:id", func(c *gin.Context) {  
    id := c.Param("id") // Récupère le paramètre 'id' de l'URL  
    c.JSON(http.StatusOK, gin.H{"user_id": id})  
})
```

Paramètres de requête (Query Strings)

```
r.GET("/search", func(c *gin.Context) {  
    query := c.Query("q") // Récupère le paramètre 'q'  
    page := c.DefaultQuery("page", "1") // Récupère 'page', avec valeur par défaut '1'  
    c.JSON(http.StatusOK, gin.H{"query": query, "page": page})  
})
```


5. Middleware dans Gin

Gin utilise les middlewares sur le principe de chaînage, similaire à `net/http`.

Exemple de middleware simple

```
func MyMiddleware() gin.HandlerFunc {  
    return func(c *gin.Context) {  
        fmt.Println("Début de la requête") // Log avant la requête  
        c.Next() // Passe au handler suivant  
        fmt.Println("Fin de la requête")   // Log après la requête  
    }  
}  
  
func main() {  
    r := gin.Default()  
  
    r.Use(MyMiddleware()) // Applique le middleware globalement  
  
    r.GET("/ping", func(c *gin.Context) {  
        c.JSON(200, gin.H{"message": "pong"})  
    })  
  
    r.Run()  
}
```

5-2-1-6

6. Exemple Complet : API CRUD Simple (Partie 1)

Structure de données et données initiales

```
package main

import (
    "github.com/gin-gonic/gin"
    "net/http"
    "strconv"
)

type User struct {
    ID int `json:"id"`
    Name string `json:"name"`
}

var users = []User{{ID: 1, Name: "Alice"}, {ID: 2, Name: "Bob"}}
```

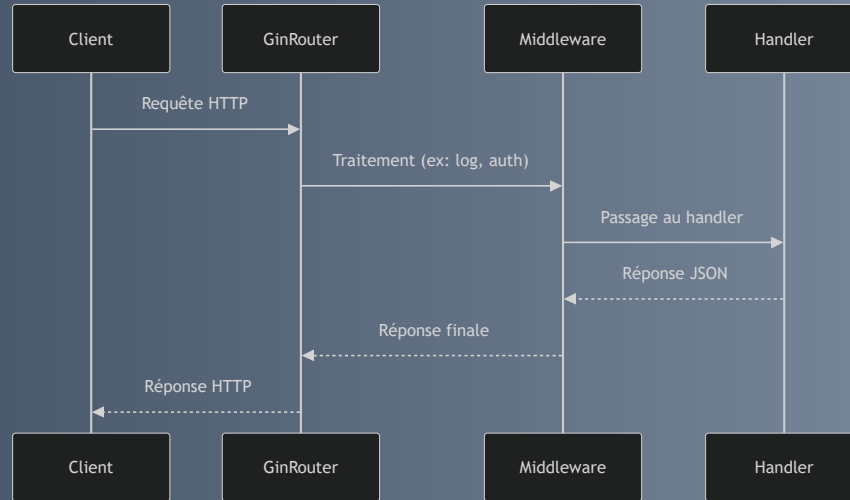
Endpoints GET et POST

```
func main() {  
    r := gin.Default()  
  
    r.GET("/users", func(c *gin.Context) { // Récupérer tous les utilisateurs  
        c.JSON(http.StatusOK, users)  
    })  
  
    r.GET("/users/:id", func(c *gin.Context) { // Récupérer un utilisateur par ID  
        id, _ := strconv.Atoi(c.Param("id"))  
        for _, u := range users {  
            if u.ID == id {  
                c.JSON(http.StatusOK, u)  
                return  
            }  
        }  
        c.JSON(http.StatusNotFound, gin.H{"message": "Utilisateur non trouvé"})  
    })  
}
```

```
// ... suite du main()
r.POST("/users", func(c *gin.Context) { // Créer un nouvel utilisateur
    var newUser User
    if err := c.ShouldBindJSON(&newUser); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }
    newUser.ID = len(users) + 1
    users = append(users, newUser)
    c.JSON(http.StatusCreated, newUser)
})

r.Run(":8080")
}
```

Flux simple d'une requête dans Gin



5-2-1-8

8. Ressources & Conclusion

Ressources pour aller plus loin

- [Gin Web Framework - Documentation officielle](#)
- [Gin by Example](#)
- [Go Web Examples - Gin](#)
- [Awesome Go - Web Frameworks](#)

Conclusion

Le framework Gin simplifie grandement la création d'API REST en Go, offrant une syntaxe claire, des performances élevées, et une modularité poussée via les middlewares. Sa popularité repose sur sa facilité d'apprentissage et son intégration fluide avec les pratiques idiomatiques de Go.

5-3-1-1

Sérialisation JSON en Go : L'Essentiel

- **JSON** est le format omniprésent pour l'échange de données dans les API web.
- Le package standard `encoding/json` en Go fournit des mécanismes puissants pour encoder et décoder les données JSON.
- Les `struct tags` associés permettent de contrôler finement la sérialisation et la désérialisation.

Transformer une Structure Go en JSON

- Pour convertir une structure Go en JSON, on utilise la fonction `json.Marshal`.

```
package main

import (
    "encoding/json"
    "fmt"
)

type User struct {
    Name string
    Age  int
}

func main() {
    u := User{Name: "Alice", Age: 30}

    data, err := json.Marshal(u)
    if err != nil {
        panic(err)
    }

    fmt.Println(string(data)) // {"Name":"Alice","Age":30}
}
```


5-3-1-3

Personnaliser la Sérialisation avec les Struct Tags

- Les **tags JSON** permettent de :
 - **Renommer** les champs dans le JSON.
 - Définir si le champ est optionnel (`omitempty`).
 - Contrôler la désérialisation.
- **Exemple de `struct tags` :**

```
type User struct {  
    Name string `json:"full_name"`  
    Age  int    `json:"age,omitempty"`  
}
```

- `full_name` sera la clé dans le JSON au lieu de `Name` .
- Si `Age` vaut 0 (valeur nulle), il ne sera pas inclus dans le JSON grâce à `omitempty` .

5-3-1-4

omitempty et Renommage : Exemples

- Démonstration de l'effet des `struct tags` lors de la sérialisation :

```
type User struct {  
    Name string `json:"full_name"`  
    Age  int    `json:"age,omitempty"`  
}  
  
func main() {  
    u := User{Name: "Bob"} // Age est 0 par défaut  
    data, _ := json.Marshal(u)  
    fmt.Println(string(data)) // {"full_name":"Bob"}  
}
```

- Dans cet exemple, le champ `Age` est omis du JSON car sa valeur par défaut est 0 et le tag `omitempty` est appliqué.

5-3-1-5

Gérer les Champs Privés et Optionnels

- **Champs non exportés** : Le package `encoding/json` encode uniquement les champs exportés (première lettre majuscule).

```
type User struct {  
    name string // non exporté  
    Age  int  
}  
// ...  
u := User{name: "Alice", Age: 25}  
data, _ := json.Marshal(u)  
fmt.Println(string(data)) // {"Age":25}
```

Gérer les Champs Privés et Optionnels

- **Champs optionnels/nullable** : Pour encoder un champ qui peut être nul ou optionnel, on peut utiliser un pointeur.

```
type User struct {  
    Name *string `json:"name,omitempty"`  
}  
// ...  
name := "Charlie"  
u := User{Name: &name} // Le champ Name est présent  
// u := User{} // Le champ Name sera omis si nil et omitempty
```

Reconstruire une Structure Go depuis JSON

- Inversement, la fonction `json.Unmarshal` permet d'extraire les données JSON et de les populer dans une structure Go.

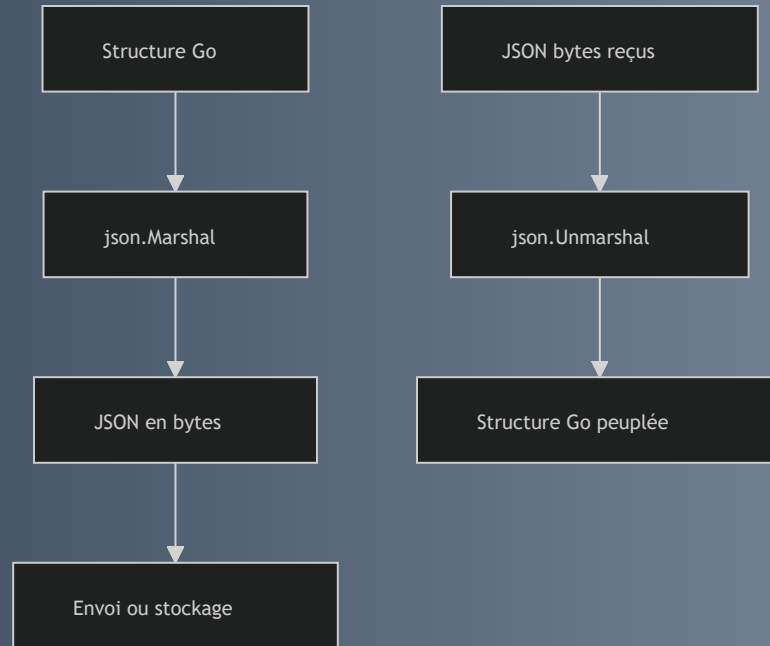
```
package main

import (
    "encoding/json"
    "fmt"
)

// Assumons User avec les tags json:"full_name" et json:"age"
type User struct {
    Name string `json:"full_name"`
    Age  int    `json:"age,omitempty"`
}

func main() {
    var u User
    jsonStr := `{"full_name":"Diana","age":22}`
    err := json.Unmarshal([]byte(jsonStr), &u)
    if err != nil {
        panic(err)
    }
    fmt.Println(u.Name, u.Age) // Diana 22
}
```

Flux de Sérialisation et Désérialisation



5-3-1-8

Maîtriser `encoding/json` : Points Clés et Ressources

- Maîtriser `encoding/json` et les `struct tags` est **indispensable** pour un développement backend fluide en Go.
- Cela permet un **contrôle précis** sur la forme et la nature des données échangées via vos API.

Ressources complémentaires :

- [Package encoding/json - Documentation officielle](#)
- [Go by Example - JSON](#)
- [Effective Go - Struct Tags](#)
- [Blog - Go JSON Best Practices](#)

5-4-1-1

Accès aux bases de données en Go : `database/sql`, `sqlx` et `GORM`

- **Introduction**

- Go interagit avec les bases de données relationnelles via le package standard `database/sql`.
- Pour simplifier et ajouter des fonctionnalités :
 - `sqlx` : une extension légère.
 - `GORM` (Go Object Relational Mapper) : un ORM complet.

5-4-1-2

database/sql : L'interface standard

- **Le package** `database/sql`
 - Fournit une interface générique pour la communication SQL.
 - Permet de se connecter, exécuter des requêtes et gérer des transactions de manière abstraite.
- **Fonctionnalités clés**
 - `sql.Open` : prépare un pool de connexions (ne l'ouvre pas immédiatement).
 - `QueryRow` : récupère une ligne unique.
 - Utilisation de placeholders : essentiel pour éviter les injections SQL.

5-4-1-3

database/sql : Ses limites

- **Mapping manuel**
 - Nécessite un mapping manuel entre les colonnes SQL et les champs des structs Go.
- **Gestion manuelle**
 - Gestion manuelle des erreurs et du scan des résultats.
- **Manque de support**
 - Pas de support natif pour les struct tags.
 - Peu d'aide pour la construction de requêtes complexes.

5-4-1-4

sqlx : L'extension légère

- **Extension de** `database/sql`
 - `sqlx` étend le package standard pour plus de commodité.
- **Fonctionnalités ajoutées**
 - **Mapping struct automatique** : grâce aux tags `db`.
 - **Méthodes pratiques** : `Get`, `Select` pour simplifier le scan des résultats.
 - **Transactions améliorées** : support plus robuste.
 - **Requêtes nommées** : pour une meilleure lisibilité.

5-4-1-5

GORM : L'ORM complet

- **Object-Relational Mapper (ORM)**
 - GORM est un ORM complet qui abstrait l'accès aux bases de données.
- **Capacités avancées**
 - **Modèles struct** : définition avec gestion des relations entre entités.
 - **Opérations CRUD** : méthodes dédiées pour créer, lire, mettre à jour, supprimer.
 - **Migrations de schéma** : intégrées pour la gestion de la base de données.
 - **Fonctionnalités étendues** : support des hooks, scopes, gestion des transactions, preload, pagination.

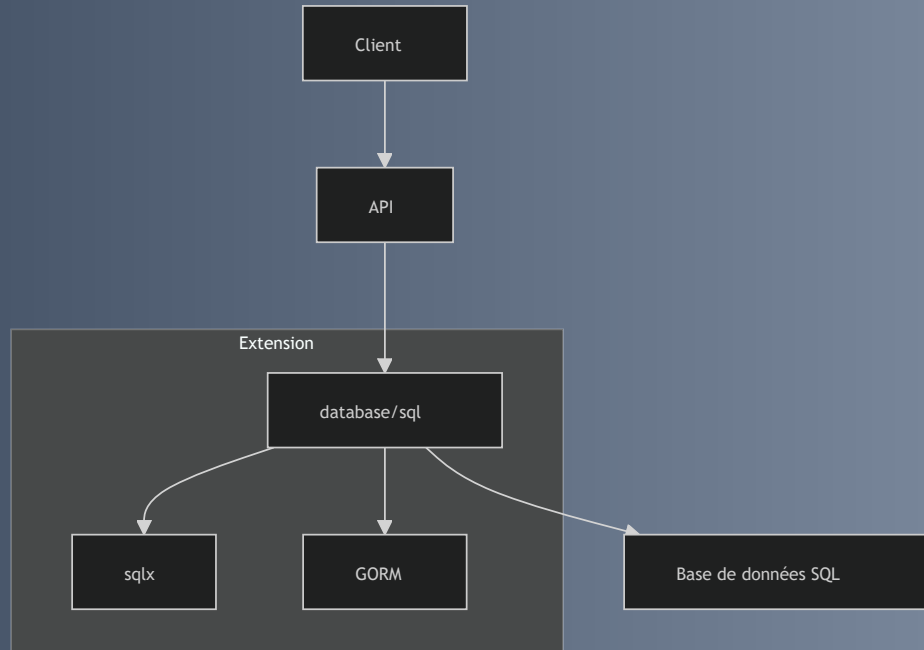
5-4-1-6

Comparaison : database/sql vs sqlx vs GORM

Aspect	database/sql	sqlx	GORM
Abstraction	Faible (bas-niveau)	Moyenne (extensions)	Élevée (ORM complet)
Mapping struct	Manuel	Automatique via tags	Automatique
Support relations	Non	Non	Oui
Migrations intégrées	Non	Non	Oui
Courbe d'apprentissage	Faible (simple)	Modérée	Plus élevée

5-4-1-7

Workflow d'accès aux bases de données en Go



Package database/sql : interface standard

Le package `database/sql` permet de se connecter, exécuter des requêtes, et gérer des transactions de manière abstraite.

Connexion et requête simple avec database/sql

```
import (  
    "database/sql"  
    "fmt"  
    _ "github.com/lib/pq" // pilote PostgreSQL  
)  
  
func main() {  
    // Connexion à la base PostgreSQL  
    db, err := sql.Open("postgres", "user=foo dbname=bar sslmode=disable")  
    if err != nil {  
        panic(err)  
    }  
    defer db.Close()
```

--> Suite -->

```
// Requête simple
var name string
err = db.QueryRow("SELECT name FROM users WHERE id = $1", 1).Scan(&name)
if err != nil {
    panic(err)
}
fmt.Println("Utilisateur:", name)
}
```

- `sql.Open` n'ouvre pas la connexion immédiatement mais prépare un pool.
- `QueryRow` récupère une ligne unique.
- Utilisation de placeholders pour éviter les injections SQL.

sqlx : extension légère de database/sql

sqlx étend `database/sql` en ajoutant notamment :

- Mapping struct automatique grâce aux tags `db`.
- Méthodes pratiques (`Get` , `Select`) pour simplifier le scan des résultats.
- Support des transactions amélioré.
- Requêtes nommées plus lisibles.

Exemple avec sqlx

```
import (
    "fmt"
    "github.com/jmoiron/sqlx"
    _ "github.com/lib/pq"
)

type User struct {
    ID    int    `db:"id"`
    Name string `db:"name"`
}
```

--> Suite -->

```
func main() {  
    db, err := sqlx.Connect("postgres", "user=foo dbname=bar sslmode=disable")  
    if err != nil {  
        panic(err)  
    }  
    defer db.Close()  
  
    var user User  
    err = db.Get(&user, "SELECT id, name FROM users WHERE id=$1", 1)  
    if err != nil {  
        panic(err)  
    }  
    fmt.Println(user)  
}
```

GORM : ORM complet avec abstraction avancée

GORM est un Object-Relational Mapper (ORM) complet qui permet :

Exemple simple avec GORM

```
import (  
    "fmt"  
    "gorm.io/driver/postgres"  
    "gorm.io/gorm"  
)  
  
type User struct {  
    ID    uint  
    Name string  
}
```

--> Suite -->

```
func main() {  
    dsn := "user=foo password=bar dbname=baz sslmode=disable"  
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})  
    if err != nil {  
        panic(err)  
    }  
  
    // Migration automatique  
    db.AutoMigrate(&User{})  
  
    user := User{Name: "Alice"}  
    db.Create(&user)  
  
    var u User  
    db.First(&u, 1)  
    fmt.Println(u.Name)  
}
```

GORM fournit un ensemble de méthodes CRUD très proches du langage naturel. Voici les principales fonctionnalités à connaître.

Création

Create

```
user := User{
    Name:  "Laurent",
    Email: "laurent@example.com",
}

result := db.Create(&user)

fmt.Println(user.ID)           // ID généré
fmt.Println(result.RowsAffected)
fmt.Println(result.Error)
```

Création multiple

```
users := []User{
    {Name: "Alice"},
    {Name: "Bob"},
}

db.Create(&users)
```

Lecture

First

Récupère le premier enregistrement selon la clé primaire.

```
var user User  
db.First(&user)
```

SQL :

```
SELECT * FROM users ORDER BY id LIMIT 1;
```

Take

Récupère un enregistrement sans tri.

```
var user User  
db.Take(&user)
```

SQL :

```
SELECT * FROM users LIMIT 1;
```


Last

Récupère le dernier enregistrement.

```
var user User
db.Last(&user)
```

SQL :

```
SELECT * FROM users ORDER BY id DESC LIMIT 1;
```

Find

Récupère plusieurs enregistrements.

```
var users []User  
db.Find(&users)
```

Recherche par clé primaire

```
var user User  
  
db.First(&user, 10)
```

SQL :

```
SELECT * FROM users WHERE id = 10 LIMIT 1;
```

Where

Filtrage.

```
db.Where("age > ?", 18).Find(&users)
```

```
db.Where("name = ?", "Laurent").First(&user)
```

Conditions avec struct

```
db.Where(&User{Name: "Laurent"}).Find(&users)
```

Select

Choisir certaines colonnes.

```
db.Select("name", "email").Find(&users)
```

Order

Tri.

```
db.Order("name asc").Find(&users)
```

```
db.Order("created_at desc").Find(&users)
```

Limit

```
db.Limit(10).Find(&users)
```

Offset

```
db.Offset(20).Limit(10).Find(&users)
```

Très utilisé pour la pagination.

Mise à jour

Save

Met à jour toute l'entité.

```
user.Name = "Nouveau nom"  
  
db.Save(&user)
```


Update

Une seule colonne.

```
db.Model(&user).Update("name", "Laurent")
```

Updates

Plusieurs colonnes.

```
db.Model(&user).Updates(User{
    Name: "Laurent",
    Age:  42,
})
```

ou :

```
db.Model(&user).Updates(map[string]interface{}{
    "name": "Laurent",
    "age":  42,
})
```

Suppression

Delete

```
db.Delete(&user)
```

ou :

```
db.Delete(&User{}, 10)
```

SQL :

```
DELETE FROM users WHERE id = 10;
```

Comptage

Count

```
var count int64

db.Model(&User{}).Count(&count)
```

Vérification d'existence

```
var user User

err := db.Where("email = ?", email).First(&user).Error

if errors.Is(err, gorm.ErrRecordNotFound) {
    fmt.Println("Utilisateur introuvable")
}
```

Relations

Preload

Chargement des relations.

```
var order Order  
  
db.Preload("Items").First(&order)
```

Équivalent :

```
SELECT * FROM orders;  
SELECT * FROM items WHERE order_id = ?;
```

Transactions

```
tx := db.Begin()

if err := tx.Create(&user).Error; err != nil {
    tx.Rollback()
    return
}

if err := tx.Create(&profile).Error; err != nil {
    tx.Rollback()
    return
}

tx.Commit()
```

Version moderne :

```
db.Transaction(func(tx *gorm.DB) error {  
  
    if err := tx.Create(&user).Error; err != nil {  
        return err  
    }  
  
    if err := tx.Create(&profile).Error; err != nil {  
        return err  
    }  
  
    return nil  
})
```


Migration automatique

AutoMigrate

```
db.AutoMigrate(&User{})
```

ou :

```
db.AutoMigrate(  
    &User{},  
    &Product{},  
    &Order{},  
)
```

Chaînage des requêtes

L'une des forces de GORM :

```
var users []User

db.
    Where("age > ?", 18).
    Order("name asc").
    Limit(20).
    Find(&users)
```

Les 15 méthodes utilisant GORM

Méthode	Utilité
Create	Insertion
First	Premier résultat
Take	Un résultat sans ordre
Last	Dernier résultat
Find	Liste de résultats
Where	Filtrage
Select	Colonnes spécifiques

--> Suite -->

Les 15 méthodes utilisant GORM

Méthode	Utilité
Order	Tri
Limit	Limitation
Offset	Pagination
Save	Mise à jour complète
Update	Une colonne
Updates	Plusieurs colonnes
Delete	Suppression
Preload	Chargement des relations

Avec ces méthodes, on couvre facilement **90 % des cas d'usage d'une API REST **.

5-4-1-8

Conclusion et Ressources

- **Le choix de l'outil**
 - Dépend des besoins du projet : complexité, performances et niveau d'automatisation.
 - `database/sql` offre un contrôle précis et manuel.
 - `sqlx` et `GORM` proposent des solutions plus expressives et productives.
- **Ressources**
 - [Package database/sql - documentation officielle](#)
 - [sqlx GitHub](#)
 - [GORM Official Site](#)
 - [Go Database/SQL Best Practices](#)
 - [Comparison of Go ORMs](#)

5-5-1-1

Tests unitaires en Go

Points clés :

- Les tests unitaires sont **fondamentaux** pour la qualité et la maintenabilité du code Go.
- `go test` : l'outil intégré de Go pour simplifier l'écriture et l'exécution des tests.
- **Table-driven tests** : une pratique idiomatique pour gérer un grand nombre de cas de test.
- **Mocking avec interfaces** : la capacité de Go à isoler les composants grâce à son typage par interfaces.

5-5-1-2

go test – Les bases des tests unitaires

- Un fichier de test porte l'extension `_test.go`.
- Les fonctions de test ont la signature `func TestXxx(t *testing.T)`.

```
package math

import "testing"

func Add(a, b int) int {
    return a + b
}

func TestAdd(t *testing.T) {
    got := Add(2, 3)
    want := 5
    if got != want {
        t.Errorf("Add(2,3) = %d; want %d", got, want)
    }
}
```

Table-Driven Tests – Organiser plusieurs cas

Principe :

- Tester une fonction avec un **ensemble de cas** définis dans une structure de données (une "table").
- Améliore la **lisibilité** et la **maintenabilité** des tests.

```
func TestAddMultiple(t *testing.T) {
    tests := []struct{
        name string
        a, b int
        want int
    }{
        {"2 plus 3", 2, 3, 5},
        {"0 plus 0", 0, 0, 0},
        {"-1 plus 1", -1, 1, 0},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got := Add(tt.a, tt.b)
            if got != tt.want {
                t.Errorf("Add(%d, %d) = %d; want %d", tt.a, tt.b, got, tt.want)
            }
        })
    }
}
```


5-5-1-5

Isolation via Mocking avec Interfaces – Mise en pratique

Fonction à tester (dépendante de l'interface) :

```
func GreetUser(svc UserService, id int) string {  
    u, err := svc.GetUser(id)  
    if err != nil {  
        return "Erreur"  
    }  
    return "Bonjour " + u.Name  
}
```

Implémentation du Mock dans le test :

```
type MockUserService struct {
    user User
    err  error
}

func (m *MockUserService) GetUser(id int) (User, error) {
    return m.user, m.err
}

func TestGreetUser(t *testing.T) {
    mockSvc := &MockUserService{
        user: User{Name: "Alice"},
        err:  nil,
    }

    got := GreetUser(mockSvc, 1)
    want := "Bonjour Alice"
    if got != want {
        t.Errorf("got %q, want %q", got, want)
    }
}
```

5-5-1-6

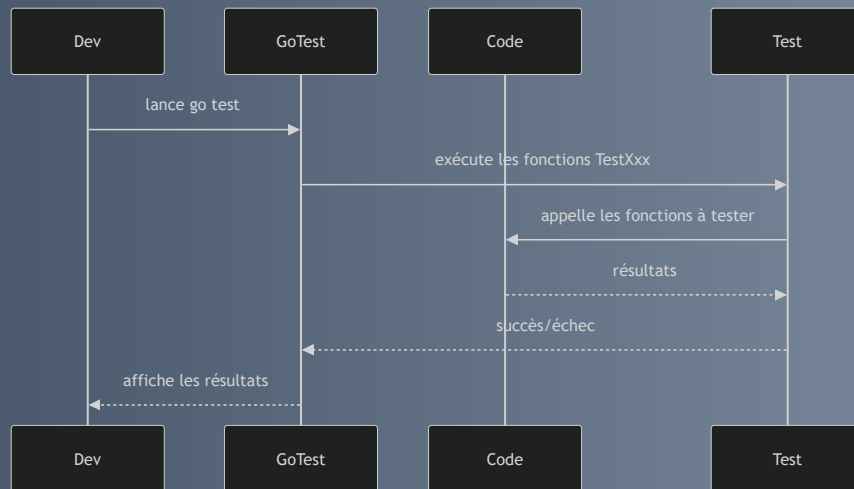
Exécution et paramètres importants de `go test`

Options utiles pour `go test` :

- `go test -v`
 - Affiche les résultats détaillés de chaque test.
- `go test -run TestName`
 - Exécute uniquement les tests dont le nom correspond à l'expression régulière `TestName`.
 - Exemple : `go test -run TestAdd`
- `go test -cover`
 - Calcule et affiche le pourcentage de couverture de code par les tests.

5-5-1-7

Processus d'un test unitaire en Go



Conclusion et Ressources

En résumé :

- L'outil `go test` est au cœur des tests unitaires en Go.
- Les **table-driven tests** garantissent une couverture complète et lisible.
- Le **mocking via interfaces** permet d'isoler les composants, assurant la fiabilité des tests.

Bénéfices :

- Couverture de code complète et maintenable.
- Fiabilité accrue des tests unitaires.
- Amélioration de la maintenabilité des projets Go.

Sources :

- [Testing package - Documentation officielle](#)
- [Go Blog - Testing techniques](#)
- [Mocking in Go with Interfaces](#)
- [Effective Go - Testing](#)

5-6-1-1

Introduction à slog (Go 1.21+)

Contexte :

- À partir de Go 1.21, la bibliothèque standard intègre le package `slog`.
- Un framework de logging structuré, léger et performant.

Objectif :

- Simplifier la production de logs riches et exploitables.
- Rester intégré dans l'écosystème standard de Go.

5-6-1-2

Pourquoi choisir slog ?

Avantages clés :

- **Logging structuré** : Logs sous forme clé-valeur, non de simples chaînes.
- **Flexibilité** : Personnalisation facile de la sortie (console, JSON, fichiers).
- **Performances** : Conçu pour minimiser l'impact sur l'exécution.
- **API simple et idiomatique** : Rappelle le package `log` classique.
- **Support natif dans la stdlib** (Go 1.21+) : Pas de dépendances externes.

5-6-1-3

Premier exemple avec slog

```
package main

import (
    "log/slog"
    "os" // Ajout de l'import os pour os.Stdout
)

func main() {
    logger := slog.New(slog.NewTextHandler(slog.NewSyncWriter(os.Stdout)))

    logger.Info("Serveur démarré",
        "port", 8080,
        "mode", "production",
    )
}
```

Sortie console attendue (format texte structuré) :

```
time=2024-06-01T12:00:00Z level=INFO msg="Serveur démarré" port=8080 mode=production
```


Handlers de sortie

Gestion de la sortie des logs :

`slog` utilise des **handlers** pour définir le format et la destination des logs.

- `NewTextHandler` : Pour des logs lisibles en console.
- `NewJSONHandler` : Pour des logs JSON, faciles à analyser par des outils externes.

Exemple avec JSON :

```
logger := slog.New(slog.NewJSONHandler(os.Stdout, nil)) // Ajout de nil pour options
logger.Info("Nouvel utilisateur créé", "username", "alice", "id", 42)
```

Sortie JSON typique :

```
{
  "time": "2024-06-01T12:00:00Z",
  "level": "INFO",
  "msg": "Nouvel utilisateur créé",
  "username": "alice",
```

Exemple d'utilisation :

```
// logger.Debug("Mode debug activé") // Nécessite un logger configuré pour Debug
logger.Warn("Mémoire basse")
// logger.Error("Erreur inattendue", "error", err) // Nécessite une variable err
```

Configuration du niveau minimal :

```
opts := &slog.HandlerOptions{Level: slog.LevelInfo} // Créer des options pour le niveau
logger := slog.New(slog.NewTextHandler(os.Stdout, opts)) // Appliquer les options
// Ce logger ignorera les logs de niveau Debug
```

5-6-1-6

Contextualisation et log structuré riche

Enrichir les logs avec des attributs fixes :

Utilisez la méthode `With` pour ajouter des attributs qui seront présents dans tous les logs d'un logger dérivé.

```
requestLogger := logger.With("request_id", "abc123", "user", "alice")

requestLogger.Info("Début traitement")
// Tous les logs via requestLogger auront "request_id" et "user" dans leurs attributs.
```

Exemple complet & Pipeline slog

Serveur HTTP avec slog :

```
package main

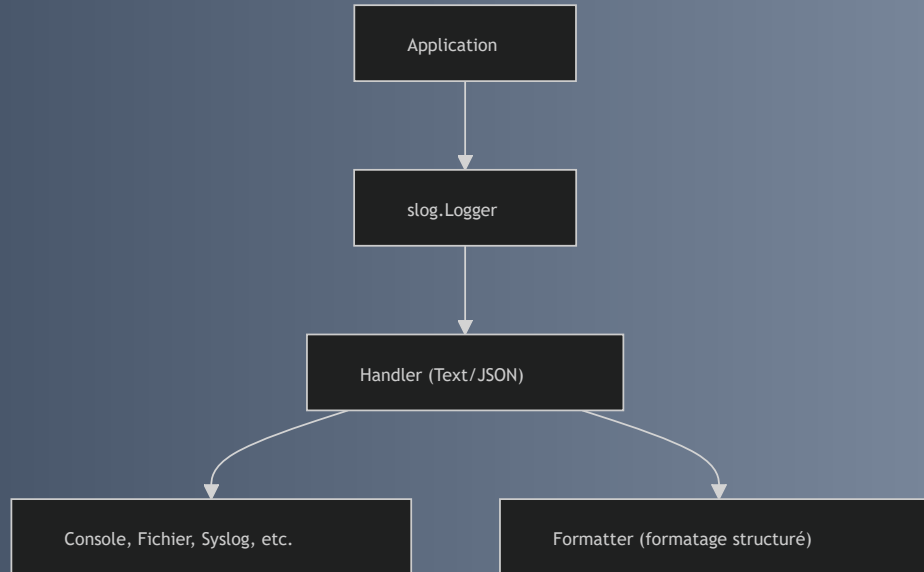
import (
    "log/slog"
    "net/http"
    "os"
)

func main() {
    logger := slog.New(slog.NewTextHandler(os.Stdout, nil))
    logger.Info("Démarrage du serveur HTTP", "port", 8080)

    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        reqLogger := logger.With("remote_addr", r.RemoteAddr, "method", r.Method, "url", r.URL.Path)
        reqLogger.Info("Requête reçue")
        w.Write([]byte("Hello, slog!"))
    })

    http.ListenAndServe(":8080", nil)
}
```

Pipeline d'un log avec slog :



5-6-1-8

Conclusion et Sources

Conclusion : Depuis Go 1.21, `slog` offre une solution moderne, intégrée au langage, pour mettre en œuvre un logging structuré performant et évolutif. Son API simple, ses handlers flexibles et ses performances en font un choix naturel pour les applications backend Go.

Sources :

- [Package slog - Documentation officielle Go 1.21+](#)
- [Go Blog – Release Notes Go 1.21](#)
- [Examples from Go source](#)
- [Talk and article: Structured logging in Go with slog](#)